# Théorie derrière la loi du $\chi^2$ et l'AFC :

## Une analyse sous le prisme des élections présidentielles 2022 en Île-de-France.

Kévin MUGUET–ROLLAND

January 2026

# Table des matières


**0 Introduction** **4**
- 0.1 Les prérequis algébriques . . . 4
  - 0.1.1 Le produit scalaire et le concept de norme associée . . . 4
  - 0.1.2 Éléments de Diagonalisation pour l'Analyse de Données . 5
- 0.2 Les prérequis probabilistiques . . . 5
  - 0.2.1 La loi de Poisson . . . 6
  - 0.2.2 La loi normale centrée réduite . . . 7
- 0.3 Conseils avant de continuer . . . 9

**1 Construction de la théorie autour de la loi du $\chi^2$** **9**
- 1.1 Construction de la loi du $\chi^2$ en probabilité . . . 9
  - 1.1.1 La fonction $\Gamma(x)$ . . . 10
  - 1.1.2 La loi $\Gamma$ . . . 11
  - 1.1.3 La loi du $\chi^2$ . . . 12
  - 1.1.4 La création de la statistique du $\chi^2$ . . . 16

**2 L'Analyse Factorielle des Correspondances (AFC)** **26**
- 2.1 La distance du $\chi^2$ . . . 27
  - 2.1.1 Espace des Profils et Centre de Gravité . . . 27
  - 2.1.2 Définition de la métrique $d_{\chi^2}$ . . . 28
  - 2.1.3 Justification Théorique : L'Équivalence Distributionnelle . 28
  - 2.1.4 Lien avec l'Inertie Totale . . . 28
- 2.2 Promis je vous la refais en français cette fois-ci . . . 28

**3 Une mise en pratique complète de l'AFC dans l'analyse des élections présidentielles 2022** **39**
- 3.1 Une analyse rapide du tableau de contingence . . . 40
- 3.2 La réduction de dimension par l'AFC . . . 45
  - 3.2.1 De l'AFC à la représentation graphique . . . 46
- 3.3 Un exemple d'implémentation de l'AFC dans une pipeline . . . 56

**4 Que faire ensuite ?** **62**

# Abstract

Je suis un étudiant d'école de commerce inscrit au Programme Grande Ecole de l'emlyon (L3), et qui suit à côté de ses études une formation autodidacte en analyse des données ainsi qu'en statistiques. Récemment, je me suis mis à l'analyse factorielle (notion abordée en début d'année de Master 1 dans un cursus universitaire), et je me suis rendu compte que je comprenais tout d'un coup beaucoup mieux comment les tests statistiques ont été fondés, chose que l'on omet souvent d'expliciter auprès d'élèves autodidactes qui se forment sur des plateformes gratuites, telles que YouTube, ou encore payantes, telles que Coursera pour les anglophones (pour ma part). Cet article n'a aucune ambition d'être un cours précis de statistiques ni même d'essayer de vous apprendre des choses ; il a seulement l'ambition de vulgariser certains concepts parfois oubliés de statistiques que je pense néanmoins important. J'essaierai tout au long d'allier à la fois rigueur potentielle, vulgarisation et intellectualisation des concepts utilisés, le tout en apportant une touche de personnalité aux explications. (Chose qui manque cruellement à des cours de mathématiques et de statistiques selon moi.) Ce présent ouvrage est aussi l'occasion pour moi de me familiariser avec l'outil LaTeX que je n'ai jamais utilisé auparavant.

Le code écrit dans cet article est retrouvable sur la même page Github sur laquellle vous avez trouvé cet article. Il existe deux notebooks : un fouilli (il suit l'ordre du coursà, et un beaucoup plus structuré dans lequel j'essaie de vous mettre des compléments de code en plus de ce cours.

# 0 Introduction

Dans cette introduction, je vais clarifier tous les éléments mathématiques qui seront utilisés (et considérés comme acquis) dans la suite de l'article.

## 0.1 Les prérequis algébriques

### 0.1.1 Le produit scalaire et le concept de norme associée

N'attendez pas de moi que je vous fasse un cours entier sur les notions de produit scalaire, ni même sur son intérêt lors des interprétations géométriques. Néanmoins, je vous rappelle la définition formelle d'un produit scalaire pour ceux qui l'auraient oubliée :

**Définition formelle** Soit $E$ un espace vectoriel réel. On appelle **produit scalaire** sur $E$ une application, notée $\langle \cdot, \cdot \rangle$, de $E \times E$ dans $\mathbb{R}$ satisfaisant les quatre propriétés suivantes pour tous vecteurs $u, v, w \in E$ et tout scalaire $\alpha \in \mathbb{R}$ :

1. **Symétrie :** $\langle u, v \rangle = \langle v, u \rangle$
2. **Bilinéarité :** $\langle \alpha u + v, w \rangle = \alpha \langle u, w \rangle + \langle v, w \rangle$
3. **Caractère positif :** $\langle u, u \rangle \geq 0$
4. **Caractère défini :** $\langle u, u \rangle = 0 \iff u = 0_E$

**Une distance algébrique** Une application $d : E \times E \to \mathbb{R}^+$ est une **distance** si elle satisfait les propriétés suivantes :

1. **Positivité :** $d(x, y) \geq 0$
2. **Séparation :** $d(x, y) = 0 \iff x = y$
3. **Symétrie :** $d(x, y) = d(y, x)$
4. **Inégalité triangulaire :** $d(x, z) \leq d(x, y) + d(y, z)$

**Lien avec le produit scalaire** Tout produit scalaire $\langle \cdot, \cdot \rangle$ définit une **norme** associée par la relation :

$$\|x\| = \sqrt{\langle x, x \rangle}$$

Cette norme permet à son tour de définir la **distance associée au produit scalaire** entre deux vecteurs $x$ et $y$ :

$$d(x, y) = \|x - y\| = \sqrt{\langle x - y, x - y \rangle}$$

En développant par bilinéarité, on obtient la relation fondamentale reliant la distance au produit scalaire :

$$d(x, y)^2 = \|x\|^2 + \|y\|^2 - 2\langle x, y \rangle$$

### 0.1.2 Éléments de Diagonalisation pour l'Analyse de Données

**Définition et Équation Caractéristique** Soit $M$ une matrice carrée symétrique réelle de taille $n$. Un vecteur non nul $u$ est un **vecteur propre** de $M$ associé à la **valeur propre** $\lambda$ si :

$$Mu = \lambda u$$

u sera dans notre cas appelé un vecteur propre associé à la valeur propre $\lambda$. Je vous rappelle qu'il existe une infinité de vecteur propre en raison de la colinéarité.

**Propriétés de la Matrice Symétrique** La symétrie de la matrice de départ garantit trois propriétés fondamentales :

- **Réalité :** Toutes les valeurs propres $\lambda_i$ sont des nombres réels.
- **Orthogonalité :** Les vecteurs propres associés à des valeurs propres distinctes sont orthogonaux : $\langle u_i, u_j \rangle = 0$ pour $i \neq j$.
- **Base Orthonormée :** Il existe une base de vecteurs propres $U = (u_1, \ldots, u_n)$ telle que chaque vecteur est de norme unité ($\|u_i\| = 1$).

**Décomposition Spectrale et Trace** La matrice $M$ peut être décomposée sous la forme :

$$M = UDU^T \quad \text{ou} \quad M = \sum_{i=1}^{n} \lambda_i u_i u_i^T$$

Une propriété cruciale pour la mesure de l'information est la conservation de la **Trace** :

$$\mathrm{Tr}(M) = \sum_{i=1}^{n} M_{ii} = \sum_{i=1}^{n} \lambda_i$$

**C'est tout** C'est à peu près tout ce que je vous demande de connaître et comprendre. Le mieux serait que vous puissiez comprendre l'implication de ces définitions et théorèmes de manière graphique. Il faudrait aussi que vous puissiez comprendre ce qu'implique l'existence d'une base dans un $\mathbb{R}$-espace vectoriel, la notion de projection (notamment orthogonale), etc... Ce présent article étant déjà assez long, je vous épargnerai l'explication de tous ces détails.

## 0.2 Les prérequis probabilistiques

Sans même aller plus loin dans l'analyse, il est nécessaire de connaitre déjà ce qu'est une probabilité. Si vous ne connaissez pas l'essentiel à savoir (théorie de la mesure, probabilité pouvant être vue comme une masse, etc...), je vous invite à regarder ce premier cours puis ce second cours proposés par l'Ecole Polytechnique sur le site de Coursera. (Un cours que j'ai fait en entier et qui pose plutôt rigoureusement les bases de la probabilité de manière exhaustive et approfondie, croyez moi !)

### 0.2.1 La loi de Poisson

**Utilité de la loi de Poisson** Avant même de débuter cette présentation de la loi de Poisson, je vais vous conter son utilité. La loi de Poisson est aussi appelée « **Loi des évènements rares** » car elle permet de modéliser le nombre de fois qu'un évènement se produit dans un intervalle de temps ou d'espace donné. Dans notre cas, ce sera la loi qui permettra de modéliser l'aléatoire contenu dans chacun des tableaux de contingence que l'on fera par la suite. (Retenez bien cela !)

**Définition de la loi de Poisson** Maintenant que son utilité ne fait plus de doute à vos yeux, définissons rigoureusement ce qu'est la loi de Poisson :

$$\forall \lambda > 0,\ X \sim \mathcal{P}(\lambda) \text{ si } \forall k \in \mathbb{N},\ \mathbb{P}(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} \tag{1}$$

La loi de Poisson a en effet été construite par le mathématicien Poisson sans doute en référence à la série exponentielle. Pour rappel, la série exponentielle est la suivante :

$$\sum_{k=0}^{+\infty} \frac{\lambda^k}{k!} = e^{\lambda}$$

Je vous laisserai alors le soin de montrer que la série entière de loi de Poisson sur son support est bien égale à 1, ce qui équivaut à dire que :

$$\sum_{k=0}^{+\infty} \mathbb{P}(X = k) = 1 \text{ avec } X \sim \mathcal{P}(\lambda)$$

**Espérance de la loi de Poisson** Ce paragraphe est le plus important sur cette loi. L'espérance de la loi de Poisson, qui est par ailleurs le résultat de la convergence asymptotique de son estimateur moyen, sera très utilisée, notamment pour définir le $\chi^2$ par la suite. Sans plus tergiverser, voici l'espérance de la loi de Poisson :

$$\mathbb{E}(X) = \lambda, \forall \lambda > 0 \tag{2}$$

Je sais que ce résultat est ma foi très difficile à retenir, et j'espère que vous vous en remettrez assez vite.

**Variance de la loi de Poisson** Comme pour toutes les lois que l'on va s'amuser à utiliser, elle est de carré intégrable, donc elle admet de surcroît une variance. Poisson aimant les lois difficiles à retenir, il n'est pas aller de main morte non plus sur le résultat de la variance :

$$\mathbb{V}(X) = \lambda,\ \forall \lambda \in \mathbb{R}_*^+. \tag{3}$$

**Exemple d'utilisation de la loi de Poisson.** La loi de Poisson dans la suite de cet article va s'utiliser comme ci-dessous.

| Departement | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 2756 | 11642 | 110118 | 6805 | 64543 | 27969 | 266632 | 5890 | 19352 | 17481 | 3663 | 6300 | 543151 |
| Val-de-Marne | 2504 | 14954 | 171409 | 8944 | 69599 | 43430 | 192427 | 8276 | 31904 | 32522 | 3730 | 9359 | 589058 |
| Val-d'Oise | 2392 | 10060 | 138166 | 9101 | 91081 | 37564 | 175666 | 6094 | 20710 | 26403 | 3352 | 8996 | 529585 |
| Essonne | 2831 | 13480 | 164503 | 11610 | 105862 | 39284 | 167310 | 7944 | 29562 | 33046 | 4306 | 15203 | 594941 |
| Hauts-de-Seine | 2306 | 13170 | 287494 | 11260 | 64812 | 62761 | 199640 | 10518 | 47103 | 62231 | 3704 | 9743 | 774742 |
| Paris | 2891 | 17267 | 372820 | 12139 | 58429 | 86088 | 317372 | 22901 | 80268 | 69564 | 5732 | 9591 | 1055062 |
| Seine-et-Marne | 3440 | 12843 | 165208 | 13797 | 155738 | 47893 | 170969 | 7853 | 26718 | 36828 | 4600 | 15042 | 660929 |
| Yvelines | 2774 | 11721 | 246062 | 13687 | 101398 | 64407 | 168585 | 9046 | 40470 | 61296 | 3963 | 13097 | 736506 |
| Total | 21894 | 105137 | 1655780 | 87343 | 711462 | 409396 | 1658601 | 78522 | 296087 | 339371 | 33050 | 87331 | 5483974 |

FIGURE 1 – Répartition des votes en Île-de-France lors des élections présidentielles de 2022.

En supposant que la variable **Candidat** et la variable **Département** sont parfaitement indépendantes l'une de l'autre, la probabilité que Nathalie Artaud soit élue en Seine-Saint-Denis ne serait pas si différente de la probabilité que Nathalie Arthaud soit en moyenne votée dans toute l'Île-de-France (qui serait si vous avez compris... le $\lambda$ théorique de la loi de Poisson car le $\lambda$ correspond à l'espérance, donc la moyenne théorique.)

Je vous épargne les calculs, mais ce $\lambda$ aurait donc pour valeur : $\lambda \approx 2168.45$. (J'assumerai tout au long de cet article que le . signifie la virgule française alors que la virgule est le séparateur de milliers.) Donc on en déduit que :

$$\mathbb{E}(X) = \mathbb{V}(X) = (21,894 * 543,151)/5,483,974 \approx 2168.45$$

Si vous avez compris, exercez-vous avec les autres cases du tableau et vérifiez lesquels, en situation d'indépendance, fournissent les mêmes résultats (ou presque), que ceux de la table de contingence. Pour ceux qui sont déjà familiés avec la notion de $\chi^2$, vous aurez remarqué que nous venons de calculer un effectif théorique. Cette notion étant primordiale pour comprendre la suite, c'est pour cela que j'ai tenu impérativement à faire ce bout de cours sur la loi de Poisson au préalable. Très peu en parle, et pourtant c'est le présupposé principal des statistiques théoriques.

### 0.2.2 La loi normale centrée réduite

Cette loi est bien évidemment **FONDAMENTALE** en analyse statistique. Je ne vous ferai pas l'affront de vous rappeler toute la théorie derrière la loi normale puisque ça n'est pas l'objectif de ce cours. Ici, on rappelera néanmoins la **densité**, l'**espérance** et la **variance** de la loi normale centrée réduite, ainsi que le théorème centrale limite qui lui est associé. L'objectif de cette sous-section

est donc d'analyser comment la loi de Poisson tend asymptotiquement vers la loi normale centrée réduite.

**Densité de la $\mathcal{N}(0,1)$** Si $X \sim \mathcal{N}(0,1)$, alors la densité de f s'exprime par :

$$\forall x \in \mathbb{R},\ f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \tag{4}$$

**Espérance de la $\mathcal{N}(0,1)$** Si $X \sim \mathcal{N}(0,1)$, alors :

$$\mathbb{E}(X) = 0 \tag{5}$$

**Variance de la $\mathcal{N}(0,1)$** Si $X \sim \mathcal{N}(0,1)$, alors :

$$\mathbb{V}(X) = 1 \tag{6}$$

**Théorème central limite** Si je vous ai rappelé toutes ces choses qui vous paraissent si anodines, c'est précisément pour ce théorème fondamental des statistiques, le **Théorème central limite**. Il se présente de la manière suivante :

Soit $(X_n)_{n\in\mathbb{N}^*}$ une suite de variables aléatoires **indépendantes** et **identiquement distribuées** de carré intégrable. On pose, de surcroît :

$$\mu = \mathbb{E}(X_1),\ \sigma^2 = \mathbb{V}(X_1),\ S_n = \sum_{k=1}^{n} X_k,\ Y_n = \frac{S_n - \mu}{\sigma}$$

Alors, $Y_n \xrightarrow{\mathcal{L}} \mathcal{N}(0,1)$. ($Y_n$ converge en loi vers la loi normale centrée réduite.)

**Problème à résoudre** Wow, ça c'est du théorème ! ! ! Ce qui va plus précisément nous intéresser, ce sera l'utilisation du théorème central limite avec la loi de Poisson. Toutefois, il faut déjà se demander s'il existe une suite de variables aléatoires $(X_n)_{n\in\mathbb{N}^*}$ telle que $S_n = \sum_{k=1}^{n} X_k$.

**Démonstration** Je partirai du principe dans cette démonstration que la notion de fonction génératrice d'une variable aléatoire discrète vous est familière. Commençons ainsi par poser :

Soit $(X_n)_{n\in\mathbb{N}^*}$ une suite de variables aléatoires de Bernoulli (de paramètre $p \in ]0,1[$) **indépendantes** et **identiquement distribuées** de carré intégrable. On pose, de surcroît :

$$\mu = \mathbb{E}(X_1) = p,\ \sigma^2 = \mathbb{V}(X_1) = p(1-p),\ S_n = \sum_{k=1}^{n} X_k \sim \mathcal{B}(n,p),\ Y_n = \frac{S_n - p}{\sqrt{p(1-p)}}$$

Rappelons aussi que :

$$\forall k \in \mathbb{N}^*,\ \forall s \in ]-1,1[,\ G_{X_k}(s) = \mathbb{E}(s^{X_k}) = (1-p) + pt$$

Donc, on en déduit, par indépendance, que :

$$G_{S_n}(s) = \prod_{k=1}^{n} G_{X_k}(s) = (1 - p + pt)^n$$

En passant à la limite, on trouve que :

$$\lim_{n \to +\infty} G_{S_n}(s) = \lim_{n \to +\infty} (1 + \frac{\lambda(s-1)}{n})^n = e^{\lambda(s-1)},\ \text{en posant } p = \frac{\lambda}{n} \text{ pour } \lambda > 0.$$

Ce résultat montre donc que la variable aléatoire $S_n$ existe au minimum comme un objet limite d'une loi binomiale. Le théorème centrale limite peut alors être utilisé pour la loi de Poisson. Nous en déduisons alors d'après le théorème limite que :

$$Y_n \xrightarrow{\mathcal{L}} \mathcal{N}(0,1). \tag{7}$$

Ce théorème sera totalement primordiale lorsqu'on va introduire la notion de résidus dans notre construction de la théorie autour de la loi du $\chi^2$.

### 0.3 Conseils avant de continuer

Comme je vous avais prévenu, ce cours présente la **VERITABLE THEORIE** probabilistique autour de la construction du $\chi^2$ et de l'AFC (Analyse Factorielle des Correspondances). Avant de continuer, vous devez donc bien maîtriser les notions de base en statistiques, en algèbre linéaire, ainsi qu'en probabilité.

## 1 Construction de la théorie autour de la loi du $\chi^2$

La loi du $\chi^2$ est une loi très connue en statistiques qui permet d'affirmer (ou infirmer) l'indépendance de deux variables aléatoires. Dans notre cas, nous allons vérifier l'indépendance de deux lois de Poisson. Néanmoins, pour arriver à ce but, nous allons faire ce que peu de cours de statistiques font, c'est-à-dire analyser la loi probabilistique du $\chi^2$ (c'est-à-dire sa densité notamment) de manière théorique, et non sous le prisme d'une table de données ou de l'utilisation d'un logiciel statistique.

### 1.1 Construction de la loi du $\chi^2$ en probabilité

Je sais que cette partie ne sera pas de tout repos puisqu'elle est profondément théorique et demande une bonne connaissance des bases en théorie probabilistique. Néanmoins, je vous demande de bien vous accrocher car une fois comprise, la loi du $\chi^2$ devient extrêmement simple, je vous l'assure !

### 1.1.1 La fonction $\Gamma(x)$

Pour pouvoir comprendre comment fonctionne la loi du $\chi^2$, nous devons déjà comprendre la fonction gamma et ses principaux résultats (dans les grandes lignes). Pour en savoir plus, regardez cette vidéo ainsi que ce cours préparé pour l'agrégation de mathématiques.

**Définition** On notera pour la suite de ce cours la fonction $\Gamma$ comme ci-dessous :

$$\forall x \in ]0, +\infty[\,,\ \Gamma(x) = \int_0^{+\infty} t^{x-1} e^{-t} dt \tag{8}$$

**Continuité** $\Gamma$ est continue sur $]0, +\infty[$. Plus généralement, $\Gamma$ est de surcroît $C^\infty$ sur $]0, +\infty[$.

**Identité fondamentale** Si la fonction $\Gamma$ est aussi convoitée par les ingénieurs, c'est à cause des deux théorèmes suivants qui lui confèrent une généralisation de la factorielle à l'ensemble des réels strictement positifs. En effet :

$$\forall x \in ]0, +\infty[\,,\ \Gamma(x+1) = x\Gamma(x). \tag{9}$$

En utilisant ce brillant résultat, on en déduit par réccurrence que :

$$\forall n \in \mathbb{N}^*,\ \Gamma(n) = (n-1)! \tag{10}$$

**Convexité** La fonction $\Gamma$ est convexe sur $]0, +\infty[$.

**Limites en $0^+$ et $+\infty$** On a que $\lim\limits_{x \to +\infty} \Gamma(x) = +\infty$ et que $\lim\limits_{x \to 0^+} \Gamma(x) = +\infty$

**Valeurs particulières de la fonction $\Gamma$** Certaines valeurs de la fonction $\Gamma$ sont connues tandis que d'autres sont plus difficiles à calculer. On a par exemple, $\Gamma(\frac{1}{2}) = \sqrt{\pi}$. Par réccurrence, vous pourriez démontrer que :

$$\forall n \in \mathbb{N},\ \Gamma(n\ + \frac{1}{2}) = \frac{(2n)!\sqrt{\pi}}{2^{2n} n!} \tag{11}$$

**Allure de la courbe** Pour terminer, je vous joins ci-dessous l'allure de la courbe de la fonction $\Gamma$ :

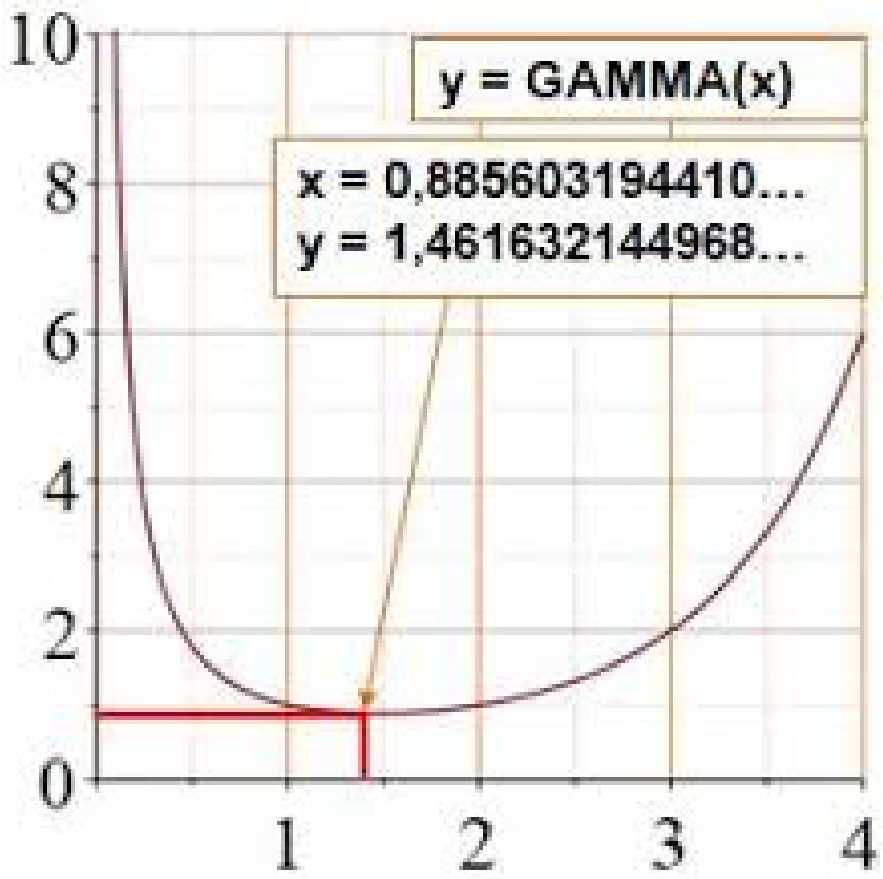



FIGURE 2 – Allure de la courbe de la fonction $\Gamma$ sur $]0, 4[$.

**NB :** La fonction $\Gamma$ possède un unique minimum (qui est donc global) comme peut nous le montrer ce graphique. (Cela provient de l'application du **Théorème de Rolle**.) De même, elle est **strictement positive**.

### 1.1.2 La loi $\Gamma$

Maintenant que nous avons (re)vu les propriétés caractéristiques de la fonction $\Gamma$, nous allons nous intéresser plus en détail à la loi de probabilité qui lui est associée. Pour plus d'information sur cette loi, je vous invite à lire cet article-ci.

**Densité de la loi $\Gamma$** On dit que $X \sim \Gamma(p, \alpha)$ avec $p > 0$ et $\alpha > 0$, si elle admet comme densité :

$$f(x) = \begin{cases} \frac{\lambda}{\Gamma(p)} e^{-\lambda x} (\lambda x)^{p-1} & si\ x > 0 \\ 0 & sinon. \end{cases} \tag{12}$$

**Espérance de la loi $\Gamma$** Comme toutes loi qui se respecte (sauf la loi de Cauchy), elle est dite intégrable. En particulier, elle admet comme espérance :

$$\mathbb{E}(X) = \frac{p}{\lambda} \tag{13}$$

**Variance de la loi $\Gamma$** Sans plus de suspens, elle est aussi de carré intégrable. Elle admet donc comme variance :

$$\mathbb{V}(X) = \frac{p}{\lambda^2} \tag{14}$$

**Fonction caractéristique** La fonction caractéristique de la loi $\Gamma$ est donnée par :

$$\forall t \in \mathbb{R},\ \Phi(t) = \left(\frac{\lambda}{\lambda - it}\right)^p \tag{15}$$

**Intégrabilité** La loi $\Gamma$ admet des moments de tout ordre.

Voici ce qui conclut notre analyse synthétique de la loi $\Gamma$. Ceci était nécessaire pour pouvoir introduire correctement la loi du $\chi^2$ puisqu'elle est en partie créée à partir de la fonction $\Gamma$.

### 1.1.3 La loi du $\chi^2$

Nous y sommes enfin ! Il est grand temps de démystifier la grande loi du $\chi^2$ qui a pu dérouter un grand nombre de statisticiens à cause de son tableau incompréhensible.

**Définition** On dit que $X \sim \chi^2(n)$ (loi du chi-deux à n degrés de liberté) si elle est à densité, et admet pour densité :

$$f(x) = \begin{cases} \frac{1}{2^{n/2}\Gamma\left(\frac{n}{2}\right)e^{-\frac{-x}{2}}x^{\frac{n}{2}-1}} & si\ x > 0 \\ 0 & sinon. \end{cases} \tag{16}$$

**Lien avec la loi $\Gamma$** Comme nous faisons les choses bien depuis le début, tu pourras aisément remarquer que la loi du $\chi^2$ n'est qu'un cas particulier de la loi $\Gamma(\frac{n}{2}, \frac{1}{2})$. Comme vous connaissez maintenant très bien cette loi, vous connaissez donc de même la loi du $\chi^2$... Félicitations !

**Propriété fondamentale** Passons aux choses sérieuses. La propriété suivante est primordiale pour comprendre l'intérêt de la loi ainsi que son usage en statistique. Supposons $(X_n)_{n\in\mathbb{N}^*}$ une suite de variables aléatoires de lois normales centrées réduites identiquement distribuées et indépendantes. On a alors que :

$$S_n = \sum_{k=1}^{n} {X_k}^2 \sim \chi(n) \tag{17}$$

Et bam, dans ta face ! Eh oui..., et pour te faire plaisir, je te propose même d'en faire la démonstration par réccurrence !

**Démonstration** Puisque ce résultat est fondamental dans ce cours, faisons-en la démonstration.

On remplace par les expressions des densités de la loi Gamma :

$$f_S(s) = \int_0^s \left(\frac{\lambda^{k_1}}{\Gamma(k_1)} x^{k_1-1} e^{-\lambda x}\right)\left(\frac{\lambda^{k_2}}{\Gamma(k_2)}(s-x)^{k_2-1} e^{-\lambda(s-x)}\right) \mathrm{d}x$$

On sort les constantes de l'intégrale et on simplifie l'exponentielle ($e^{-\lambda x}e^{-\lambda s+\lambda x} = e^{-\lambda s}$) :

$$f_S(s) = \frac{\lambda^{k_1+k_2}}{\Gamma(k_1)\Gamma(k_2)} e^{-\lambda s} \int_0^s x^{k_1-1}(s-x)^{k_2-1}\,\mathrm{d}x$$

Pour résoudre cette intégrale, on effectue le changement de variable $u = \frac{x}{s}$, d'où $x = su$ et $\mathrm{d}x = s\,\mathrm{d}u$ :

$$\begin{aligned}\int_0^s x^{k_1-1}(s-x)^{k_2-1}\,\mathrm{d}x &= \int_0^1 (su)^{k_1-1}(s-su)^{k_2-1} s\,\mathrm{d}u \\ &= s^{k_1-1+k_2-1+1}\int_0^1 u^{k_1-1}(1-u)^{k_2-1}\,\mathrm{d}u \\ &= s^{k_1+k_2-1}\int_0^1 u^{k_1-1}(1-u)^{k_2-1}\,\mathrm{d}u\end{aligned}$$

On reconnaît ici la **fonction Bêta** : $B(k_1,k_2) = \int_0^1 u^{k_1-1}(1-u)^{k_2-1}\,\mathrm{d}u$. En utilisant la relation $B(k_1,k_2) = \frac{\Gamma(k_1)\Gamma(k_2)}{\Gamma(k_1+k_2)}$, on obtient :

$$f_S(s) = \frac{\lambda^{k_1+k_2}}{\Gamma(k_1)\Gamma(k_2)} e^{-\lambda s} s^{k_1+k_2-1} \frac{\Gamma(k_1)\Gamma(k_2)}{\Gamma(k_1+k_2)}$$

En simplifiant les termes $\Gamma(k_1)\Gamma(k_2)$, il reste :

$$f_S(s) = \frac{\lambda^{k_1+k_2}}{\Gamma(k_1+k_2)} s^{(k_1+k_2)-1} e^{-\lambda s}$$

On identifie bien la densité d'une loi $\Gamma(k_1 + k_2, \lambda)$.

Démontrons maintenant que le carré d'une loi normale centrée réduite peut s'apparenter à une loi du $\chi^2$.

Soit $X \sim \mathcal{N}(0,1)$ de densité $f_X(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$.

Soit $Y = X^2$. Pour $y > 0$, la fonction de répartition $F_Y$ est :

$$F_Y(y) = P(X^2 \leq y) = P(-\sqrt{y} \leq X \leq \sqrt{y})$$

$$F_Y(y) = \int_{-\sqrt{y}}^{\sqrt{y}} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \, \mathrm{d}x = 2 \int_0^{\sqrt{y}} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \, \mathrm{d}x$$

On dérive par rapport à $y$ pour obtenir la densité $f_Y(y)$ :

$$f_Y(y) = \frac{\mathrm{d}}{\mathrm{d}y} F_Y(y) = 2 \cdot \frac{1}{\sqrt{2\pi}} e^{-(\sqrt{y})^2/2} \cdot \frac{\mathrm{d}}{\mathrm{d}y}(\sqrt{y})$$

$$f_Y(y) = \frac{2}{\sqrt{2\pi}} e^{-y/2} \cdot \frac{1}{2\sqrt{y}} = \frac{1}{\sqrt{2\pi}} y^{-1/2} e^{-y/2}$$

En utilisant la relation $\Gamma(1/2) = \sqrt{\pi}$, on peut réécrire :

$$f_Y(y) = \frac{(1/2)^{1/2}}{\Gamma(1/2)} y^{1/2-1} e^{-y/2}$$

On reconnaît la densité d'une loi $\Gamma(k = 1/2, \lambda = 1/2)$, également appelée loi du $\chi^2(1)$.

On souhaite démontrer par réccurrence la proposition suivante :

$$\mathcal{P}(n) : \ \forall n \in \mathbb{N}^*, \ X_n = \sum_{k=1}^{n} {X_k}^2 \sim \chi^2(n)$$

Initialisation : Soit $n = 1$, d'après les résultats précédent, on a bien que $X_1^2 \sim \mathcal{N}(0,1)$ donc $X_1^2 \sim \chi^2(1)$

Hérédité : Supposons qu'il existe un entier $n \geq 1$ tel que la proposition $\mathcal{P}(n)$ soit vraie. C'est-à-dire que $S_n = \sum_{k=1}^{n} X_k^2 \sim \chi^2(n)$.

Considérons maintenant la somme au rang $n+1$ :

$$S_{n+1} = \sum_{k=1}^{n+1} X_k^2 = S_n + X_{n+1}^2$$

Par hypothèse de récurrence et d'après les résultats précédents :

— $S_n \sim \chi^2(n)$, ce qui équivaut à $S_n \sim \Gamma\left(\frac{n}{2}, \frac{1}{2}\right)$

— $X_{n+1}^2 \sim \chi^2(1)$, ce qui équivaut à $X_{n+1}^2 \sim \Gamma\left(\frac{1}{2}, \frac{1}{2}\right)$

Les variables $(X_k)$ étant indépendantes, $S_n$ et $X_{n+1}^2$ le sont également. D'après la propriété d'additivité des lois Gamma (démontrée par convolution), la somme de deux variables Gamma indépendantes de même paramètre d'intensité $\lambda = \frac{1}{2}$ suit une loi Gamma dont le paramètre de forme est la somme des paramètres de forme :

$$S_{n+1} \sim \Gamma\left(\frac{n}{2} + \frac{1}{2}, \frac{1}{2}\right)$$

$$S_{n+1} \sim \Gamma\left(\frac{n+1}{2}, \frac{1}{2}\right)$$

Par définition, cela correspond à une loi du Chi-deux à $n+1$ degrés de liberté :

$$S_{n+1} \sim \chi^2(n+1)$$

La proposition $\mathcal{P}(n+1)$ est donc vérifiée.

Conclusion : La propriété est initialisée pour $n = 1$ et est héréditaire. Par principe de récurrence, elle est vraie pour tout $n \in \mathbb{N}^*$ :

$$\sum_{k=1}^{n} X_k^2 \sim \chi^2(n)$$

Et bah voilà, bravo ! C'est une bonne chose de faite, on a réussit à caractériser cette loi. Les choses qui arrivent sont beaucoup plus théoriques et corsées, ce n'est que du bonus, pour mieux comprendre la notion de degrés de liberté. Je ne passerai pas plus de temps à expliquer les concepts que j'utilise car cela sort énormément de l'ambition de ce petit cours d'introduction approfondie aux statistiques.

**Quelques éléments de démonstration de la convergence vers le $\chi^2$ dans le cas de l'imparfaite indépendance** Soit $Z = (z_1, z_2, \ldots, z_n)^T$ le vecteur des résidus de Pearson, où $n = L \times K$. Dans le cadre du test d'indépendance, sous l'hypothèse nulle $H_0$, ce vecteur suit asymptotiquement une loi normale multidimensionnelle centrée : $Z \sim \mathcal{N}(0, \Sigma)$. Si les cases étaient indépendantes, la matrice de covariance $\Sigma$ serait la matrice identité $I_n$. Cependant, les contraintes marginales imposent des corrélations entre les résidus.

**Propriétés de la matrice de covariance** On démontre que dans le cas du test du $\chi^2$, la matrice de covariance $\Sigma$ est une matrice de projection orthogonale. Elle possède deux propriétés fondamentales :

— Elle est symétrique : $\Sigma^T = \Sigma$

— Elle est idempotente : $\Sigma^2 = \Sigma$

D'après les propriétés des matrices de projection, ses valeurs propres ne peuvent être que 0 ou 1. Le nombre de valeurs propres égales à 1 correspond exactement au rang de la matrice, noté $k$.

**Décomposition spectrale et changement de base** Puisque $\Sigma$ est symétrique, elle est diagonalisable dans une base orthonormée. Il existe une matrice orthogonale $P$ telle que $\Sigma = PDP^T$, où $D$ est une matrice diagonale contenant $k$ fois la valeur 1 et $(n-k)$ fois la valeur 0. Définissons un nouveau vecteur aléatoire $Y = P^T Z$. Par linéarité, $Y$ suit également une loi normale centrée, et sa matrice de variance-covariance est :

$$Var(Y) = P^T \Sigma P = D$$

Cela signifie que les composantes $y_1, \ldots, y_k$ sont des variables normales centrées réduites $\mathcal{N}(0,1)$ indépendantes, tandis que les autres composantes $y_{k+1}, \ldots, y_n$ sont nulles (puisque leur variance est nulle).

**Conclusion sur la loi du $\chi^2$** La statistique globale du $\chi^2$ est la norme au carré du vecteur $Z$. Puisque la matrice $P$ est orthogonale, elle préserve la norme (isométrie) :

$$X^2_{obs} = \|Z\|^2 = \|Y\|^2 = \sum_{i=1}^{k} y_i^2 + \sum_{j=k+1}^{n} 0^2 = \sum_{i=1}^{k} y_i^2$$

Par définition, la somme des carrés de $k$ variables normales indépendantes suit une loi du $\chi^2$ à $k$ degrés de liberté. Comme nous avons établi précédemment que le nombre de contraintes indépendantes réduit le rang de l'espace à $k = (L-1)(K-1)$, la statistique converge bien vers une loi $\chi^2((L-1)(K-1))$.

### 1.1.4 La création de la statistique du $\chi^2$

**Cadre** Définissons le cadre de notre analyse pour mieux comprendre ce que l'on fait. Pour rappel, nous nous plaçons dans le cadre d'analyse de deux variables dites qualitatives, et nous voulons conclure quant à leur indépendance supposée. (Ce sera le motif de notre hypothèse $H_0$.) Pour ce faire, nous devons construire une métrique statistique qui pourra ensuite être insérée dans une fonction de densité de probabilité (qui s'avèrera être celle du $\chi^2$). Pour arriver à cela, nous analyserons donc une table de contingence (cf. Figure 1) dont on supposera que chacune des cases suit une loi de Poisson de paramètre l'effectif attendu. (Nous définirons plus bas ce que cela signifie clairement.)

**Définition de l'effectif attendu** Sous l'hypothèse nulle $H_0$ d'indépendance entre les deux variables qualitatives, l'effectif attendu (ou théorique) représente la valeur que l'on devrait observer dans une case si le hasard seul dictait la répartition des données. Pour une cellule située à l'intersection de la ligne $i$ et de la colonne $j$, cet effectif, que nous noterons $E_{ij}$, est obtenu en calculant le produit du total de la ligne $i$ par le total de la colonne $j$, le tout divisé par l'effectif total $N$ de l'échantillon. Mathématiquement, cela revient à projeter la distribution marginale globale sur chaque sous-groupe local :

$$E_{ij} = \frac{\text{Total Ligne}_i \times \text{Total Colonne}_j}{N}$$

**Justification du modèle de Poisson** L'utilisation de la loi de Poisson pour modéliser ces effectifs repose sur un schéma de convergence probabiliste. Initialement, si l'on considère une case précise du tableau, le nombre d'individus qu'elle contient suit une loi binomiale $\mathcal{B}(n, p)$. Ici, $n$ représente le nombre total d'individus de la ligne (par exemple, le nombre de votants dans un département) et $p$ représente la probabilité pour un individu d'appartenir à la colonne (par exemple, voter pour un candidat spécifique). Dans le cadre d'études électorales, le nombre d'électeurs $n$ est très grand tandis que la probabilité $p$ est très faible. Sous l'hypothèse d'indépendance des électeurs, cette loi binomiale converge vers une loi de Poisson de paramètre $\lambda = np$ (résultat que nous avons démontré précédemment à l'aide des fonctions génératrices).

Il est alors fondamental de comprendre que ce produit $np$ est, par définition, l'effectif attendu $E_{ij}$. En effet, sous l'hypothèse d'indépendance $H_0$, la probabilité $p$ d'appartenir à une colonne est la même pour tout le monde et se calcule par le ratio : $\text{Total Colonne}_j/N$. En multipliant cette probabilité globale par l'effectif local du département ($n = \text{Total Ligne}_i$), on obtient le nombre de personnes que l'on s'attend statistiquement à trouver dans la case. Ainsi, le paramètre $\lambda$ de notre loi de Poisson et l'effectif attendu $E_{ij}$ sont une seule et même valeur. Comme l'espérance d'une loi de Poisson est égale à son paramètre, $E_{ij}$ constitue la "moyenne théorique" autour de laquelle les observations réelles doivent fluctuer si le hasard est le seul facteur à l'œuvre.

**Exemple pratique et application au cas Arthaud** Afin d'illustrer ce cadre théorique, nous pouvons appliquer ces calculs aux données de Nathalie Arthaud en Seine-Saint-Denis. Nous cherchons à vérifier si son score observé ($O_{\text{Seine-Saint-Denis, Arthaud}} = 2,756$) s'écarte significativement de la moyenne théorique. En considérant les 543 151 votants du département et la proportion nationale de voix de la candidate ($21,894/5,483,974$), nous calculons le paramètre de notre loi de Poisson locale :

$$\lambda_{\textbf{Seine-Saint-Denis, Arthaud}} = \frac{543,151 \times 21,894}{5,483,974} \approx 2,168.45$$

Cette valeur de $2,168.45$ constitue l'effectif attendu sous l'hypothèse $H_0$. L'écart observé par rapport à cette moyenne servira de base à la construction de notre

résidu statistique, permettant de quantifier si l'anomalie constatée en Seine-Saint-Denis relève d'une fluctuation aléatoire ou d'une spécificité territoriale marquée.

**Reposition du problème** Nous voilà bien avancé puisque nous avons pu remettre un cadre plus théorique à ce que nous voulions analyser. Vu que nos effectifs observés et attendus sont en relation avec une loi de Poisson,qui est elle même grâce au théorème centrale limite en relation avec une loi normale centrée réduite, vous voyez peut-être comment la loi du $\chi^2$ va s'opérer. En effet, il va falloir maintenant que l'on force l'utilisation du théorème centrale limite pour se ramener à la loi normale centrée réduite.

**Définition du résidus comme écart normalisé** Si l'on récupère notre effectif observé $O_{ij}$ comme défini précédemment et qu'on essaie de le normaliser, on obtiendrait la chose suivante en considérant la loi $X \sim \mathcal{P}(E_{ij})$ :

$$\frac{O_{ij} - \mathbb{E}(X)}{Std(X)}$$

Ce qui est équivalent à écrire :

$$\frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}}}$$

On écrira donc par la suite que :

$$Z_{ij} = \frac{O_{ij} - E_{ij}}{\sqrt{E_{ij}}} \tag{18}$$

Il devient alors évident par le théorème central limite que $Z_{ij}$ converge en loi vers la loi normale centrée réduite. On appelera ainsi ceci le résidus standardisé de Pearson.

**Explications** Si vous souhaitez mieux comprendre cette formule, laissez-moi vous en toucher deux ou trois mots. Tout d'abord, l'écart $O_{ij} - E_{ij}$ n'est rien d'autre qu'un écart à la moyenne. Seulement, chacune des cases de notre tableau de contingence peut se retrouver avec des "unités" différentes, et donc des variances différentes. En effet, là où Jean-Luc Mélanchon a reçu des millions de votes au travers de l'Île-de-France, Phillipe Poutou n'en a reçu que quelques milliers. Si l'on ne prend pas compte de ces écarts, les résidus obtenus seraient difficilement analysables car ils ne seraient pas à la même échelle. La division par l'écart type (standard deviation en anglais, d'où l'écriture Std) permet ainsi de ramener chacune des observations à la même échelle, rendant par la même occassion l'analyse d'autant plus simple. De même, cela nous permet de tricher et d'utiliser le théorème centrale limite pour nous ramener plus tard à la loi du $\chi^2$, nous avons donc dégoté le jackpot sur ce coup !

**Contribution de Pearson au $\chi^2$** Il est grand temps d'en finir ! Mettons un terme à cette attente. Je sais que vous mourrez d'envie de retrouver la loi du $\chi^2$ ! Comme chacun des $Z_{ij}$ suit une loi normale centrée réduite, si on les met tous au carré, puis on les somme (supposons qu'il y en ait $L \times K$), on tombera alors sur la loi du $\chi^2$ en vertu des démonstrations faites plus haut. Dans le mille Emile. Plus sérieusement, on appelle **contribution de Pearson** au $\chi^2$ la statistique suivante :

$$C_{ij} = Z_{ij}^2 \tag{19}$$

Et on notera, en supposant que le tableau d'effectif est composé de L lignes et de K colonnes, que :

$$\chi^2 = \sum_{i=1}^{L}\sum_{j=1}^{K} C_{ij} = \sum_{i=1}^{L}\sum_{j=1}^{K} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \tag{20}$$

Essayons maintenant de comprendre pourquoi en réalité le degré de liberté n'est pas $L \times K$

**Démonstration des degrés de liberté** Pour démontrer le nombre de degrés de liberté d'une table de contingence de taille $L \times K$, il faut s'intéresser au nombre de contraintes linéaires indépendantes qui pèsent sur les cases du tableau. Initialement, si l'on considère chaque case $O_{ij}$ comme une variable pouvant varier librement, nous disposons d'un système à $L \times K$ dimensions. Cependant, dans le cadre du test du $\chi^2$, nous travaillons à totaux marginaux fixés. Chaque total de ligne et chaque total de colonne constitue une équation de contrainte qui vient réduire la dimension de l'espace dans lequel les données peuvent fluctuer.

**Le décompte des contraintes indépendantes** Listons précisément ces contraintes. Nous avons d'une part $L$ équations correspondant aux totaux des lignes : $\sum_{j=1}^{K} O_{ij} = n_{i\cdot}$ pour chaque ligne $i$. D'autre part, nous avons $K$ équations correspondant aux totaux des colonnes : $\sum_{i=1}^{L} O_{ij} = n_{\cdot j}$ pour chaque colonne $j$. Au total, cela semble représenter $L + K$ contraintes. Toutefois, ces équations ne sont pas toutes linéairement indépendantes. En effet, la somme des totaux des lignes est rigoureusement égale à la somme des totaux des colonnes (les deux égalant l'effectif total $N$). Cette redondance implique que la dernière contrainte peut toujours être déduite des autres. Le nombre de contraintes indépendantes effectives est donc de $(L + K - 1)$.

**Calcul final par soustraction de dimension** Le nombre de degrés de liberté ($ddl$) est défini mathématiquement comme la dimension de l'espace de départ moins le nombre de contraintes linéaires indépendantes imposées au système. Le calcul se développe de la manière suivante :

$$ddl = (L \times K) - (L + K - 1)$$

En réorganisant les termes de cette expression, nous obtenons :

$$ddl = LK - L - K + 1$$

Cette expression polynomiale se factorise alors parfaitement sous la forme suivante :

$$ddl = (L-1)(K-1)$$

**Lien avec la statistique de test** Cette démonstration explique pourquoi, bien que nous calculions et sommions les contributions de l'intégralité des $L \times K$ cases, le résultat final se comporte statistiquement comme une somme de carrés de seulement $(L-1)(K-1)$ variables normales indépendantes. En algèbre linéaire, on dirait que la statistique du $\chi^2$ est une forme quadratique dont le rang est $(L-1)(K-1)$. C'est ce paramètre, et non le nombre total de cases, qu'il faut utiliser pour identifier la loi de probabilité de référence et déterminer la p-value du test. La différence avec le degré de liberté que nous avions défini de manière théorique plus haut avec la loi du $\chi^2$ réside notamment dans le fait que les variables ne sont pas parfaitement indépendantes, et que des lignes de totaux sont imposées.

**Inversibilité de la FDR de la loi $\Gamma$** Comme vous pouvez vous en douter, le plus dur est derrière nous. Nous venons de définir explicitement et de manière rigoureuse comment la statistique de test du $\chi^2$ a été créée. Néanmoins, il nous reste encore à définir la p-value ainsi que la récupération des quantiles, comme nous le faisons pour n'importe quel test statistique. Si on se réfère au signe de la fonction de densité de la loi $\Gamma$, on en déduit que sa primitive est strictement croissante de $[0,1] \to \mathbb{R}^+$, elle est donc bijective et admet en particulier une fonction réciproque que l'on notera $F^{-1}$ par la suite. Cette réciproque sera très utile pour calculer le quantile.

**La p-value** Maintenant que nous avons défini la statistique du $\chi^2$, il est grand temps de trouver une méthode statistique nous permettant de savoir si l'hypothèse $H_0$ que l'on a posé est absurde. Pour ce faire, nous définissons le risque de première espèce que nous noterons $\alpha$. Pour les exemples, j'utiliserai la valeur la plus commune de $\alpha$, c'est-à-dire 0.05. J'utiliserai pour l'exemple une loi de $\chi^2$ à un faible degré de liberté, c'est-à-dire 1. Cela représente, si vous l'avez compris, le cas d'un tableau de contingence à deux lignes et deux colonnes.

Ci-dessous se trouve un code python permettant d'afficher le tracé de la loi du $\chi^2$ à 1 degré de liberté, ainsi que la notion de risque de première espèce pour un $\alpha = 0.05$.

```python
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats

X = np.linspace(0,10,100) #On crée cent abscisses créées de manière équiréparties sur [0,10[
ddf = 1 #le degré de liberté est mis à 1

fdr_chi = stats.chi2.cdf(X,ddf)
densite_chi = stats.chi2.pdf(X,ddf)

alpha = 0.05
seuil = stats.chi2.ppf(1 - alpha, df=1) # Calcul automatique du quantile (3.84)
X_fill = np.linspace(seuil, 10, 100)
Y_fill = stats.chi2.pdf(X_fill, df=1)

fig,ax = plt.subplots(nrows=1,ncols=2,sharex= True,figsize=(10,5))

ax[0].plot(X,densite_chi)
ax[0].set_title("Densité de la loi du \n $\chi^2$ à 1 ddl." )
ax[0].fill_between(X_fill, Y_fill, color='red', alpha=0.3, label=f'Zone de rejet (alpha={alpha})')
ax[0].legend()

ax[1].plot(X,fdr_chi)
ax[1].set_title("Fonction de répartition de la loi \n du $\chi^2$ à 1 ddl." )


plt.show()
```

FIGURE 3 – Utilisation de la librairie Scipy.

Le graphique résultant montre en rouge l'air correspondant à la p-value du test statistique.

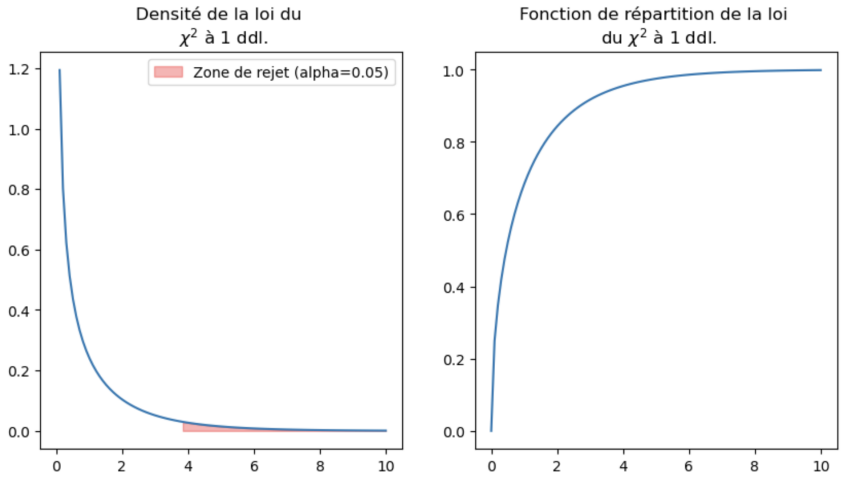



FIGURE 4 – Graphiques sur la loi du $chi_2$ et compréhension de la p-value.

Ainsi, le graphique nous montre bel et bien que la fonction de répartition de la loi du $chi^2$ à 1 ddl est bijective. Intuitivement, on comprend rapidement que la p-value correspond grossièrement à la probabilité que notre hypothèse $H_0$ soit complètement bidon. (C'est une mesure de la solidité de l'hypothèse $H_0$ puisqu'elle détermine à quel point le hasard permet d'expliquer les résultats obtenus, ici liés à l'indépendance puisqu'on a construit le test de $\chi^2$ autour de cette notion centrale.) Dans notre cas, nous considérerons que cette zone de rejet admet un risque de premier espèce $\alpha = 0.05$, c'est-à-dire, $\alpha = 5\%$. Nous n'accepterons donc pas plus de 5% d'explication de la réfutation de l'hypothèse $H_0$ comme totalement soumise au hasard. (Bien entendu, j'essaie de vulgariser au plus ici la notion, en réalité, c'est bien plus compliqué que cela.)

**Illustration** Tentons d'imager au mieux ce concept de p-value avec une comparaison qui m'avait bien aidé à mes débuts en statistiques. Laissez-vous emporter par la justice, et devenez du jour au lendemain un procureur de la République. Comme la justice française le proclame, chaque accusé bénéficie du principe du "bénéfice du doute", ou encore dénommée "présomption d'innocence". L'accusé ici est le **hasard**. En tant que procureur, vous devez donc respecter l'innocence de ce cher accusé en premier chef-lieu. Vous posez donc votre premier verdict, $H_0$ : "On suppose que Monsieur l'accusé **Hasard** est innocent". Tout ce qu'on a pu observer jusqu'ici est entièrement créé par lui et lui seul, et cela lui revient de droit ! Néanmoins, vous êtes procureur, et votre but est de détruire les fausses manigances du hasard. Tout n'est pas créé par lui, et il tente de s'approprier les créations potentielles d'autres personnes qui n'ont pas été créditées, telles que des structures aléatoires précises. (Quel sale gosse quand même.) Toutefois, toute accusation doit se faire par l'administration d'une preuve donc en tant que procureur, vous vous devez de les apporter.

C'est donc ce que vous faites en calculant votre petite statistique du $\chi^2$ qui calcule dans votre cas les écarts à l'indépendance. Vous savez que plus cette preuve évolue en votre sens (plus la valeur du $\chi^2$ augmente), plus votre accusation se retrouvera légitimée aux yeux du juge ! Néanmoins, vous ne lui avez pas encore mis le coup de grâce ! L'épée de damoclès qui se retrouve au dessus de la tête de notre accusé **Hasard** sévira lorsque la p-value l'aura dictée, puisqu'elle et elle seule a le pouvoir de dire « Si le hasard était vraiment innocent, quelle est la probabilité que les preuves contre lui soient le fruit d'une simple coïncidence ? ». Et oui, les mensonges existent... mais les faits aussi ! Imaginons que la p-value est de 0.50 (50 %) : C'est un énorme doute. Le hasard peut très bien avoir produit ce résultat par pur coup de chance, il ne ment donc a priori pas. Le juge (le scientifique) dit : "Relâchez le hasard, la preuve est trop faible." Si la p-value est de 0,01 (1 %) : Le doute est infime. Si le hasard était réellement innocent, il n'y aurait qu'1% de chances d'observer une preuve aussi forte contre lui. Le juge dit : "Le doute n'est plus raisonnable. Je condamne le hasard, il y a un effet réel." ? Dans cet exemple, le seuil de rejet $\alpha$ s'inscrit comme le code pénal des statistiques : qu'est-ce qui légitime un doute raisonnable ? Finalement, si tu fixes un seuil $\alpha$ trop élevé (par exemple 20 %), tu vas condamner le hasard beaucoup trop souvent alors qu'il était innocent. C'est l'erreur judiciaire : tu affirmes qu'un résultat est "exceptionnel" alors que c'était juste un coup de chance. Bon dieu que les bavures policières existent, mais en statistiques, on essaiera de les éviter au maximum ! Comme en justice, condamner ne signifie pas détenir la vérité absolue, mais juger que le doute n'est plus raisonnable, n'oubliez donc jamais que vos tests peuvent aussi vous mentir, notamment si vous ne respectez pas les suppositions théoriques derrières. (cf. **Théorème de Slutsky** par exemple.)

**Mise en action via Python** Vous savez donc tout sur le test d'indépendance du $\chi^2$ en long, en large et en travers. Nous allons donc maintenant regarder comment nous pouvons intégrer notre nouvel outil via le puissant outil **SciPy**.

```python
import pandas as pd #Outil permettant de simuler un tableur comme sous Excel
from scipy.stats import chi2_contingency

#Récupération du tableau de données sur les élections présidentielles de 2022 en Île de France.
data_raw = pd.read_excel("election_2022_FR.xlsx")
data = data_raw.copy()
dpt = data["Departement"].values
data = data.set_index("Departement")

#On rajoute la ligne et la colonne totale à ce tableau de contingence.
sum_temp = data.sum(axis=0)
data.loc["Total"] = sum_temp

sum_temp = data.sum(axis=1)
data["Total"] = sum_temp

#Ici, on garde uniquement le tableau de contingence car il nous sera utile pour la suite (lors de l'AFC)
data_cont = data.copy().iloc[:-1,:-1]

#Finalement, le voici notre test de chi2.
chi2, p, dof,expected = chi2_contingency(data_cont)
print(f'Chi2 score {chi2} | p-value {p} | DoF {dof }')
```

```
Chi2 score 345250.998352663 | p-value 0.0 | DoF 77
```

FIGURE 5 – Test de $\chi^2$ avec SciPy

Comme vous le voyez, **SciPy** nous donne automatiquement la valeur statistique du $\chi^2$ ($\approx 345,251$), la valeur de la p-value ($\approx 0$ %) ainsi que le degré de liberté (77). En prime, vous avez même la matrice des effectifs attendus (les $E_{ij}$) dans la variable **expected**. Comme nous faisons notre analyse avec un seuil de rejet $\alpha = 0.05$, il est clair que $H_0$ est rejetable. Attention, ici les effectifs sont dans l'ordre du million et le degré de liberté n'est que de 77... Cela signifie que le plus souvent, le degré de liberté ne rend pas la courbure du $\chi^2$ assez smooth, ce qui nous amène à avoir des p-value ridiculement petites. En effet, il est quasiment impossible d'avoir une parfaite indépendance entre des variables catégorielles sur un échantillon aussi large quand bien même elles seraient bien indépendantes puisque le bruit naturel viendrait s'ajouter petit à petit et s'amplifier. (Note : le degré de liberté est censé couper la chique à ce chaos, ce "bruit")

**Quelques indicateurs reliés au $\chi^2$** Pour terminer en beauté, je vais vous donner quelques indicateurs qui permetteront de tirer la meilleure partie de cette statistique. Au menu : inertie et $\nu$ de Cramer. Je ne reviendrai pas sur les notions de résidus et de contribution de Pearson au $\chi^2$ car je les estime acquises puisqu'on les a déjà vues.

Je tiens à vous dire aussi que l'inégalité suivante est une inégalité fondamentale pour comprendre comment analyser les différents indicateurs :

$$\chi^2 \leq n \times (\min(L, C) - 1) \tag{21}$$

où n correspond à l'effectif total du tableau de contingence, L le nombre de lignes et C le nombre de colonnes.

Sans plus tarder, je vous présente l'inertie (un gros merceau lorsqu'on va faire de l'AFC ensuite) :

$$Inertie = \phi^2 = \frac{\chi^2}{n} \tag{22}$$

L'inertie mesure donc la "quantité d'information" ou l'écart global à l'indépendance, mais elle est affranchie de la taille de l'échantillon, ce qui la rend un peu plus interprétable. Néanmoins, si l'on se réfère à l'inéquation fondamentale (21), vous remarquerez sans doute que cette métrique reste difficilement interprétable pour un $min(L, C) - 1 > 1$ puisqu'elle ne variera pas toujours entre et 1. Pour éviter ce problème, le mathématicien Cramer nous a pondu cette "masterclass" :

$$\nu = \sqrt{\frac{\phi^2}{\min(L, C) - 1}} \tag{23}$$

En gros, il a juste standardisé l'inertie pour qu'elle soit entre 0 et 1 quelque soit le degré de liberté de la loi du $\chi^2$, puis il la met sous la racine carré pour qu'elle cesse d'être un carré, ce qui la rend encore plus interprétable. On a aussi que :

0 : Indépendance totale. 1 : Liaison parfaite.

**Analogie** Ce que je vais vous écrire maintenant est la deuxième raison (la première étant la comparaison judiciaire) de pourquoi je voulais vous écrire ces pages. Je sais que beaucoup de personnes sont familières avec les notions de variance, de covariance, d'espérance, etc... Ces notions sont seulement attribuables à des données quantitatives ! Je trouve trop de personnes heureuses de faire par exemple un PCA sur des variables qualitatives alors qu'en théorie, elle ne s'applique qu'aux variables quantitatives. Arrêtez de croire que les notions de variances et de covariances existent pour des variables qualitatives. (Quand bien même nous les aurions Bernoullisées puis inscrites dans un tableau de contingence.) Cependant, comme je suis sympathique, je vous annonce que toutes les notions que nous avons étudiées jusqu'à maintenant sont les équivalent des outils utilisés pour les variables quantitatives !.

Pour commencer, notre contribution de Pearson au $\chi^2$ est au tableau de contingence ce que la Somme des Carrés des Ecarts (SCE) est à la variance. C'est une mesure de "dispersion des données" par rapport à un modèle stable (ici d'indépendance). Ensuite, vous savez sûrement que $\mathbb{V}(X)$ : $\frac{SCE(X)}{n}$, et bien ici on a de manière analogue que $\phi^2 = \frac{\chi^2}{n}$.

Finalement, on remarquera aussi que le coefficient de corrélation linéaire de Pearson ainsi que le V de Cramer reposent sur le même principe : on divise une mesure de liaison par une mesure de dispersion totale. Pour Pearson ($r$) : On prend la Covariance (la variabilité partagée) et on la normalise par les écarts-types.

$$r = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

Pour le V de Cramer : On prend la statistique du $\chi^2$ (qui mesure l'écart à l'indépendance) et on la normalise par l'effectif total ($n$) et la dimension du tableau.

$$V = \sqrt{\frac{\chi^2/n}{\min(k-1, p-1)}}$$

Néanmoins, là où Pearson peut varier de -1 jusqu'à 1, le V de Cramer ne le peut pas, puisqu'une corrélation inverse d'indépendance n'aurait aucun sens dans le cadre de notre analyse... Enfin voilà, si vous avez compris le parallèle et les rapprochements que l'on puisse faire entre ces deux théories, il sera d'autant plus simple de l'assimiler, la comprendre, puis de la réutiliser en pratique. En ce qui concerne cette partie de construction de la théorie "simple" autour du test de $chi^2$, elle est dors et déjà terminée. J'espère vous avoir aiguillé sur certaines choses, j'espère aussi peut être vous avoir appris d'autres choses, ou bien même vous avoir fait révisé ! Cependant, ne partez pas tout de suite car... les choses intéressantes vont bientôt débuter. Il est grand temps de développer ce pour quoi ce cours a réellement été construit, c'est-à-dire l'**Analyse Factorielle des Correspondances** (AFC).

# 2 L'Analyse Factorielle des Correspondances (AFC)

Vous aurez compris que jusque maintenant, nous pouvons dire si deux variables qualitatives sont indépendantes ou non. Pourtant, cela ne semble toujours pas assez satisfaisant comme résultat. En effet, si les deux variables ne sont pas indépendantes, nous voulons maintenant savoir pourquoi.

Le test du $\chi^2$ et ses indicateurs dérivés (comme le $V$ de Cramer) sont des outils dits **synthétiques** : ils rendent un verdict global mais souffrent de deux limites majeures pour l'analyste.

## Les limites du test du $\chi^2$

- **Une vision globale mais aveugle :** Le $\chi^2$ résume toute la complexité d'un tableau de contingence en un seul chiffre. Il nous indique que les variables sont liées, mais il est incapable de préciser la **nature** de ce lien. Il additionne les écarts à l'indépendance sans distinguer quelles modalités s'attirent ou se repoussent.
- **La saturation cognitive :** Dès que le tableau dépasse quelques lignes et colonnes (par exemple 10 secteurs géographiques et 5 candidats), l'esprit humain ne peut plus détecter les structures de ressemblance à la simple lecture des effectifs. On perçoit le bruit global, mais on perd le signal structurel.

## L'AFC : "Photographier" la dépendance

L'Analyse Factorielle des Correspondances intervient pour transformer ce verdict statistique en une représentation géométrique. Son objectif est de **dé-**

**composer l'inertie totale** (notée $\phi^2 = \frac{\chi^2}{n}$) pour la projeter sur des axes visuels.

> *Si le $\chi^2$ est la preuve qu'une relation existe, l'AFC en dessine la géographie.*

L'AFC apporte trois valeurs ajoutées fondamentales :

1. **Visualisation des proximités :** Sur un plan factoriel, deux modalités sont proches si elles présentent des profils similaires. Par exemple, deux départements seront spatialement proches s'ils partagent des comportements électoraux identiques.
2. **Hiérarchisation de l'information :** L'AFC classe les écarts à l'indépendance par ordre d'importance. Le premier axe du graphique (l'axe factoriel 1) capture la part la plus importante de l'inertie, révélant ainsi la tendance lourde des données.
3. **Synthèse multidimensionnelle :** Elle permet de réduire un tableau complexe de $L$ lignes et $C$ colonnes à un nuage de points en deux ou trois dimensions, rendant les anomalies et les structures immédiatement identifiables.

En somme, l'AFC permet de passer d'une statistique de décision (le $\chi^2$) à une **statistique d'exploration**. On ne se contente plus de rejeter l'hypothèse nulle $H_0$, on explore la structure de l'hypothèse alternative $H_1$.

## 2.1 La distance du $\chi^2$

Si je vous ai convaincu de continuer à lire cet articlen tant mieux. Néanmoins, pour que l'on arrive à bon port, il va falloir transformer notre bon vieil ami le $\chi^2$ en un outil mathématiques plus général et compréhensible. Pour ce faire, nous allons sortir l'artillerie lourde : l'algèbre linéaire. Plus précisément, nous allons utiliser les structures d'espaces vectoriels normés, je vous demande donc (comme précisé dans le préambule de cet article) d'être à l'affût sur les notions de distances. Si ceci est le cas, laissez moi vous présenter ce "banger", la distance du $\chi^2$.

### 2.1.1 Espace des Profils et Centre de Gravité

Soit un tableau de contingence $K$ de dimensions $(I \times J)$. On note $f_{ij} = n_{ij}/n$ les fréquences relatives, $f_{i.}$ les marges lignes (masses des lignes) et $f_{.j}$ les marges colonnes.

On définit le **profil de la ligne** $i$ comme le vecteur des fréquences conditionnelles :

$$\mathbf{P_i} = \left( \frac{f_{i1}}{f_{i.}}, \frac{f_{i2}}{f_{i.}}, \dots, \frac{f_{iJ}}{f_{i.}} \right)$$

Le centre de gravité $G$ du nuage des profils-lignes est le vecteur des fréquences marginales des colonnes :

$$\mathbf{G} = (f_{.1}, f_{.2}, \dots, f_{.J})$$

### 2.1.2 Définition de la métrique $d_{\chi^2}$

La distance entre deux profils-lignes $i$ et $i'$ est une distance euclidienne pondérée par l'inverse des poids des colonnes. La forme quadratique associée à cette métrique est définie par la matrice diagonale $\mathbf{D_J^{-1}} = \text{diag}(1/f_{.j})$.

La distance du $\chi^2$ au carré entre deux profils s'écrit :

$$d^2_{\chi^2}(i, i') = \|\mathbf{P_i} - \mathbf{P_{i'}}\|^2_{D_J^{-1}} = \sum_{j=1}^{J} \frac{1}{f_{.j}} \left( \frac{f_{ij}}{f_{i.}} - \frac{f_{i'j}}{f_{i'.}} \right)^2 \tag{24}$$

### 2.1.3 Justification Théorique : L'Équivalence Distributionnelle

Le choix de cette métrique est dicté par le **principe d'équivalence distributionnelle** (Axiome de Benzécri). Ce principe stipule que si deux colonnes $j$ et $j'$ possèdent des profils identiques, leur fusion en une colonne unique ne doit pas modifier la distance entre les lignes.

L'utilisation du terme de pondération $1/f_{.j}$ assure cette stabilité et permet de donner une importance équitable aux modalités rares, évitant ainsi que les colonnes à fort effectif ne saturent l'information géométrique.

### 2.1.4 Lien avec l'Inertie Totale

L'objectif de l'AFC est de projeter ce nuage sur des axes qui capturent le maximum de dispersion. L'inertie totale $\mathcal{I}$ du nuage est la somme des distances au carré des profils au centre de gravité $G$, pondérée par les masses $f_{i.}$ :

$$\mathcal{I} = \sum_{i=1}^{I} f_{i.} d^2_{\chi^2}(\mathbf{P_i}, \mathbf{G}) \tag{25}$$

En développant cette expression, on retrouve la relation fondamentale avec la statistique de test :

$$\mathcal{I} = \sum_{i=1}^{I} \sum_{j=1}^{J} \frac{(f_{ij} - f_{i.}f_{.j})^2}{f_{i.}f_{.j}} = \frac{\chi^2}{n} = \phi^2 \tag{26}$$

*Ainsi, l'inertie totale d'un nuage de profils en AFC est rigoureusement égale à l'inertie globale du tableau (phi-deux), reliant ainsi l'approche probabiliste du test à l'approche géométrique de l'analyse de données.*

## 2.2 Promis je vous la refais en français cette fois-ci

Je vous rappelle qu'on étudie un tableau de contingence qui est de la forme suivante :

| Departement | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 2756 | 11642 | 110118 | 6805 | 64543 | 27969 | 266632 | 5890 | 19352 | 17481 | 3663 | 6300 | 543151 |
| Val-de-Marne | 2504 | 14954 | 171409 | 8944 | 69599 | 43430 | 192427 | 8276 | 31904 | 32522 | 3730 | 9359 | 589058 |
| Val-d'Oise | 2392 | 10060 | 138166 | 9101 | 91081 | 37564 | 175666 | 6094 | 20710 | 26403 | 3352 | 8996 | 529585 |
| Essonne | 2831 | 13480 | 164503 | 11610 | 105862 | 39284 | 167310 | 7944 | 29562 | 33046 | 4306 | 15203 | 594941 |
| Hauts-de-Seine | 2306 | 13170 | 287494 | 11260 | 64812 | 62761 | 199640 | 10518 | 47103 | 62231 | 3704 | 9743 | 774742 |
| Paris | 2891 | 17267 | 372820 | 12139 | 58429 | 86088 | 317372 | 22901 | 80268 | 69564 | 5732 | 9591 | 1055062 |
| Seine-et-Marne | 3440 | 12843 | 165208 | 13797 | 155738 | 47893 | 170969 | 7853 | 26718 | 36828 | 4600 | 15042 | 660929 |
| Yvelines | 2774 | 11721 | 246062 | 13687 | 101398 | 64407 | 168585 | 9046 | 40470 | 61296 | 3963 | 13097 | 736506 |
| Total | 21894 | 105137 | 1655780 | 87343 | 711462 | 409396 | 1658601 | 78522 | 296087 | 339371 | 33050 | 87331 | 5483974 |

FIGURE 6 – Représentation d'un tableau de contingence

**Profil-ligne et Profil-colonne** Je pense que vous l'aurez compris, mais les lignes représentent ici les Départements et les colonnes représentent les candidats. J'appellerai le profil-ligne de la ligne i et de la colonne j comme ceci : $L_{ij} = \frac{n_{ij}}{n_i}$ avec $n_{ij}$ l'effectif croisé que l'on retrouve dans la $i^e$ ligne et la $j^e$ colonne de la table de contingence et $n_i$ le total de la ligne i. Par exemple, $L_{1,1} = \frac{2,756}{543,151} \approx 0.005$ et $L_{3,3} = \frac{138,166}{529,585} \approx 0.26$ et ainsi de suite. De manière analogue, nous appelons le profil-colonne $C_{ij} = \frac{n_{ij}}{nj}$, où par soucis de simplicité on notera $n_j$ comme le total de la colonne j. (Là où si c'était i, on parlerait de la ligne i, donc faites attention à ce jeu d'écriture simplificateur mais qui pourrait s'avérer trompeur pour certains.) Par exemple, $C_{11} = \frac{2,756}{21,894} \approx 0.13$ et $C_{3,3} = \frac{138,166}{1,655,780} \approx 0.08$. Pour aller plus vite, nous ferons par la suite un programme Python qui vous automatisera le calcul. En attendant, je vous incite à faire tous les calculs **TANT QUE** vous n'avez pas interiorisé par coeur les notions de profil-ligne et de profil-colonne.

**Profil-ligne moyen et Profil-colonne moyen** Ils sont encore plus simple à calculer que les profil-lignes et les profil-colonnes. Je noterai $L_i$ le profil-ligne moyen de la ligne i et $L_i = \frac{n_j}{n}$. Par exemple, $L_1 = \frac{21,894}{5,483,974} \approx 0.1$ et $L_3 = \frac{1,655,780}{5,483,974} \approx 0.30$. De manière analogue, je noterai $C_j$ le profil-colonne moyen de la colonne j et $C_j = \frac{n_i}{n}$. Par exemple, $C_1 = \frac{543,151}{5,483,974} \approx 0.1$ et $C_3 = \frac{529,585}{5,483,974} \approx 0.1$.

**Construction des tableaux en Python** Automatisons via les codes Pythons ci-dessous le calcul des tableaux d'analyses de profil-ligne et de profil-colonne.



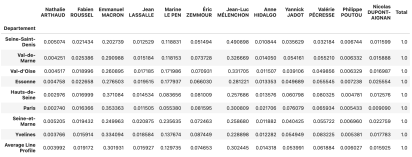

(a) Génération des profil-lignes




(b) Génération des profil-colonnes

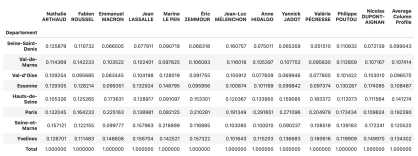
FIGURE 7 – Voici comment générer en Python les tableaux des profil-lignes et des profil-colonnes

Et bien sûr, voici les résultats obtenus :



| Département | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 0.005074 | 0.021434 | 0.202739 | 0.072529 | 0.118831 | 0.051494 | 0.406696 | 0.070844 | 0.035629 | 0.032184 | 0.006744 | 0.011599 | 1.0 |
| Val-de-Marne | 0.004251 | 0.025396 | 0.200988 | 0.075184 | 0.118753 | 0.073728 | 0.326669 | 0.074193 | 0.054161 | 0.055210 | 0.006332 | 0.015888 | 1.0 |
| Val-d'Oise | 0.004517 | 0.019996 | 0.200695 | 0.077165 | 0.171996 | 0.079501 | 0.331705 | 0.071507 | 0.039106 | 0.049856 | 0.006329 | 0.016067 | 1.0 |
| Essonne | 0.004758 | 0.022658 | 0.276503 | 0.081675 | 0.177937 | 0.066030 | 0.281221 | 0.073353 | 0.049669 | 0.066545 | 0.007238 | 0.025564 | 1.0 |
| Hauts-de-Seine | 0.002976 | 0.016999 | 0.371084 | 0.074554 | 0.083696 | 0.081009 | 0.257666 | 0.051576 | 0.060798 | 0.080325 | 0.004781 | 0.012576 | 1.0 |
| Paris | 0.002740 | 0.016366 | 0.353363 | 0.071505 | 0.055380 | 0.061595 | 0.300809 | 0.057196 | 0.076079 | 0.065934 | 0.005403 | 0.009090 | 1.0 |
| Seine-et-Marne | 0.005205 | 0.019432 | 0.249963 | 0.020875 | 0.235635 | 0.072463 | 0.256660 | 0.011882 | 0.040425 | 0.055722 | 0.006660 | 0.022769 | 1.0 |
| Yvelines | 0.003766 | 0.015914 | 0.334094 | 0.016584 | 0.157674 | 0.087449 | 0.228898 | 0.012282 | 0.054949 | 0.080225 | 0.005381 | 0.017783 | 1.0 |
| Average Line Profile | 0.003992 | 0.019172 | 0.301931 | 0.071927 | 0.129735 | 0.074653 | 0.302445 | 0.074318 | 0.053991 | 0.061884 | 0.006527 | 0.015925 | 1.0 |

(a) Profil-lignes



| Département | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Average Column Profile |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 0.125879 | 0.110732 | 0.066505 | 0.077911 | 0.090719 | 0.066318 | 0.160757 | 0.075011 | 0.065359 | 0.051510 | 0.110832 | 0.072139 | 0.099043 |
| Val-de-Marne | 0.114369 | 0.142233 | 0.103522 | 0.102401 | 0.097625 | 0.106083 | 0.116018 | 0.105397 | 0.107752 | 0.096830 | 0.102859 | 0.107167 | 0.107414 |
| Val-d'Oise | 0.109254 | 0.095985 | 0.063445 | 0.104198 | 0.128019 | 0.091755 | 0.105912 | 0.077609 | 0.069944 | 0.077800 | 0.101422 | 0.103010 | 0.096670 |
| Essonne | 0.129305 | 0.126214 | 0.090551 | 0.102924 | 0.146795 | 0.099996 | 0.100674 | 0.137169 | 0.099642 | 0.097374 | 0.130287 | 0.114083 | 0.116487 |
| Hauts-de-Seine | 0.155326 | 0.152305 | 0.171453 | 0.138927 | 0.091207 | 0.153201 | 0.120367 | 0.133950 | 0.156565 | 0.183372 | 0.112073 | 0.111564 | 0.141274 |
| Paris | 0.132045 | 0.164233 | 0.225163 | 0.139991 | 0.062125 | 0.210291 | 0.161349 | 0.201631 | 0.271096 | 0.204979 | 0.173434 | 0.109824 | 0.182392 |
| Seine-et-Marne | 0.167121 | 0.122155 | 0.098777 | 0.167963 | 0.218899 | 0.116995 | 0.103080 | 0.100010 | 0.090237 | 0.108618 | 0.139183 | 0.172241 | 0.126520 |
| Yvelines | 0.126701 | 0.111483 | 0.148606 | 0.156704 | 0.142521 | 0.157322 | 0.101643 | 0.115233 | 0.136883 | 0.180616 | 0.119909 | 0.149970 | 0.134302 |
| Total | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 |

(b) Profil-colonnes

FIGURE 8 – Résultats donnés par Python

**Qu'est-ce qu'on calcule concrètement là ?** Pour votre plus grand plaisir, je vais essayer de vous donner la signification de tous les coefficients que j'ai calculé à la main pour que vous comprenez au mieux ce que l'on est en train de calculer :). Premièrement, l'analyse en ligne répond à la question suivante : "Dans chaque département, quelle est la répartition des proportions de vote en Île de France selon les différents participants à l'élection présidentielle de 2022 ?" Le profil-ligne moyen, quant à lui, se réfère à la proportion totale que représente les votes du candidat j parmi l'entièreté des votes de l'Île de France. On remarquera que celui qui a la proportion la plus élevée était donc Jean-Luc Mélenchon, suivi de très près par Emmanuel Macron ! D'autre part, le profil-colonne répond à la question suivante : "Pour chaque candidat, quelle est la répartition des proportions de vote selon chaque département d'Île de France ?" Le profil-colonne moyen, quant à lui, se réfère à la proportion de vote globale dans chaque département, qu'importe le candidat. On remarquera par exemple que sur cent personnes dans les Yvelines, 15 personnes ont voté Macron tandis que sur cent personnes d'Île de France, 13 personnes ont voté dans une urne située dans les Yvelinnes. Ainsi, pour l'AFC, le choix de la lecture en profil-ligne ou en profil-colonne dépendra naturellement des questions et des analyses que vous voudrez dresser puisque les deux méthodes amèneront au même résultat. Dans un premier temps, nous nous contenterons de dresser la théorie pour le profil-ligne, car il est plus intuitif d'étudier le comportement électoral propre à chaque territoire. En effet, cela nous permet d'analyser la « signature politique » de chaque département en observant comment les suffrages s'y répartissent entre les différents candidats.

**Des visualisations en vrac** Le principe même de l'ACP est de fournir des visualisations. Commençons donc par faire quelques petites visualisations faciles à comprendre pour débuter. Il va s'agir de représenter de manière intuitive et compréhensible le résultat des tableaux de profil-ligne et de profil-colonne. Ce faisant, je vous propose de lancer ces deux codes Python suivants :

```
ax1 = line_profile_per.iloc[:,:-1].plot(kind="barh", stacked=True,figsize=(7,7) )
ax1.legend(title="Candidates", bbox_to_anchor=(1.05, 1), loc='upper left')
ax1.set_title("Line profiles of French Elections (%)")
ax1.set_xlabel("%")
```

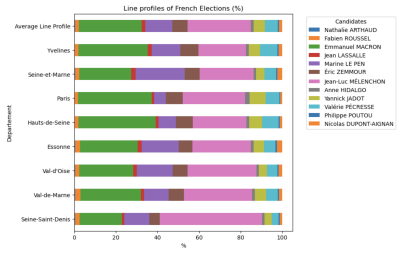
(a) Génération de l'histogramme empilé des profil-lignes

```
ax2 = column_profile_per.iloc[:-1,:].T.plot(kind="barh",stacked=True,figsize=(7,7))
ax2.legend(title= "Departement",bbox_to_anchor=(1.05,1))
ax2.set_title("Column Profiles of French Elections (%)")
ax2.set_xlabel("%")
```

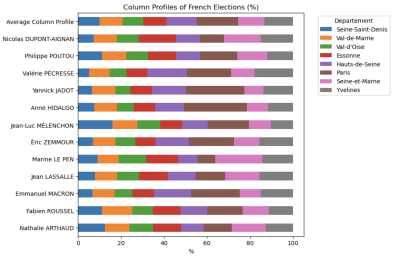
(b) Génération de l'histogramme empilé des profil-colonnes

FIGURE 9 – Comment générer un histogramme empilé en Python via matplotlib.pyplot

Et voici en avant-première le résultat que je trouve tout à fait remarquable (je ne sais pas pour vous) :




(a) Profil-lignes




(b) Profil-colonnes

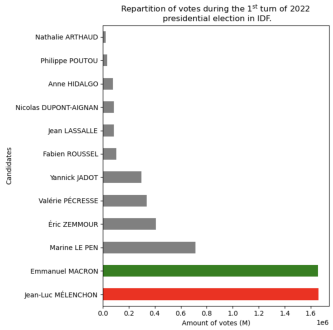
FIGURE 10 – Résultats renvoyés par Python

Je vous avais prévenu, ça en jette, mais vous n'êtes bien sûr pas au bout de vos surprises. En bonus, je vous lache aussi cette bombe, c'est cadeau :

```python
ax3 = data.loc["Total",:].iloc[:-1].sort_values(ascending=False).plot(kind="barh",color=["Red","Green"] + ["Grey"]*10,figsize=(7,7))
ax3.set_ylabel("Candidates")
ax3.set_xlabel("Amount of votes (M)")
ax3.set_title("Repartition of votes during the $\mathrm{1^{st}}$ turn of 2022 \n presidential election in IDF.")

plt.tight_layout()
plt.savefig("2022_presidential_election_barh")
```

(a) Code Python




(b) Résultat renvoyé par Python

FIGURE 11 – Répartition des votes en Île de France selon les différents candidats présidents lors des élections présidentielles de 2022

Il semblerait que Macron et Mélenchon ont roulé sur toute la concurrence... Deux frères, deux fauves !

**Distance entre un profil et le profil-moyen associé** Maintenant qu'on a bien retravaillé les notions de profils, on va voir comment on peut calculer l'écart entre un profil et son profil-moyen associé. En réalité, on va se retrouver à calculer une distance de $\chi^2$ mais d'une manière un tout petit peu plus différente qu'avant. Laissez-moi vous définir ce qu'est la distance de $\chi^2$ entre un profil-ligne total $LT_i$ (une ligne de notre tableau) et le profil-moyen associé $LM$ (la ligne "Total") :

$$d^2(LT_i, LM) = \sum_{j=1}^{J} \frac{(L_{ij} - LM_j)^2}{LM_j} \tag{27}$$

Vous remarquerez que j'ai adapté les notations ici : $L_{ij}$ représente l'élément du profil-ligne dans la case de coordonnée $ij$, tandis que $LM_j$ est la proportion moyenne de la colonne $j$. Cette quantité n'est pas une invention arbitraire ; elle découle directement de la statistique structurelle.

## De la loi de Bernoulli au carré des résidus

**L'échelle de l'électeur et du département** Le vote d'un individu suit une loi de Bernoulli, et à l'échelle d'un département (ou d'une CSP), le cumul des voix suit une loi Multinomiale. Pour de grands effectifs, cette distribution converge vers une loi Normale. C'est ce qui nous permet de définir le **résidu de Pearson** $Z_{ij}$, qui mesure l'écart normalisé entre l'observé et le théorique :

$$Z_{ij}^2 = \frac{(n_{ij} - n_{i.}LM_j)^2}{n_{i.}LM_j}, \text{ avec } n_{i.} \text{ qui représente le total de la ligne i} \tag{28}$$

**L'émergence de la distance par unité de masse** En factorisant par la masse de la ligne $n_{i.}$, on peut extraire le terme qui nous intéresse pour notre géométrie :

$$Z_{ij}^2 = n_{i.} \times \frac{(L_{ij} - LM_j)^2}{LM_j} \tag{29}$$

On voit alors que si l'on somme ces résidus sur toute une ligne (pour tous les candidats $j$), on retombe sur notre distance $d^2$ multipliée par le poids de la catégorie :

$$\sum_{j=1}^{J} Z_{ij}^2 = n_{i.} \times \underbrace{\sum_{j=1}^{J} \frac{(L_{ij} - LM_j)^2}{LM_j}}_{d^2(LT_i, LM)} \tag{30}$$

**Statut statistique de la distance du $\chi^2$** Il est important de souligner que la distance du $\chi^2$ entre un profil-ligne $LT_i$ et le profil moyen $LM$,

$$d^2(LT_i, LM) = \sum_{j=1}^{J} \frac{(L_{ij} - LM_j)^2}{LM_j},$$

n'est pas, en elle-même, une variable aléatoire suivant une loi du $\chi^2$. Il s'agit avant tout d'un objet **géométrique**, mesurant l'écart entre deux profils dans l'espace des profils-lignes muni de la métrique du $\chi^2$.

En revanche, la statistique globale du test d'indépendance,

$$\chi^2 = \sum_{i=1}^{I} \sum_{j=1}^{J} Z_{ij}^2,$$

où

$$Z_{ij}^2 = \frac{(n_{ij} - n_{i.} LM_j)^2}{n_{i.} LM_j},$$

suit asymptotiquement une loi du $\chi^2$ à $(I-1)(J-1)$ degrés de liberté sous l'hypothèse d'indépendance.

Cette statistique peut se réécrire comme une somme pondérée de distances du $\chi^2$ entre chaque profil-ligne et le profil moyen :

$$\chi^2 = \sum_{i=1}^{I} n_{i.}\, d^2(LT_i, LM).$$

Ainsi, la distance du $\chi^2$ intervient comme un élément fondamental dans la décomposition géométrique d'une statistique probabiliste qui, elle, suit une loi du $\chi^2$.

Voici l'intégralité du passage, structuré et prêt à être inséré dans ton document LaTeX :

Extrait de code

**Lien direct avec l'inertie** Comme nous avons recréé la statistique du $\chi^2$, nous pouvons aussi recréer $\phi^2$, puis le $V$ de Cramer, etc. Attention, la statistique obtenue est l'inertie totale ! Nous pouvons aussi ne calculer que l'inertie d'une seule ligne (inertie partielle) :

$$Inertie(L_i) = P_i \times d^2(LT_i, LM) \tag{31}$$

où $P_i$ représente le poids de la ligne, c'est-à-dire sa masse calculée par le rapport $n_{i.}/n$. Cette mesure nous permet de pondérer la "singularité" d'un profil par son importance numérique dans l'échantillon.

En sommant ces inerties pour l'ensemble des lignes du tableau, nous obtenons l'**Inertie Totale** du nuage de points :

$$Inertie_{Totale} = \sum_{i=1}^{I} P_i \times d^2(LT_i, LM) = \sum_{i=1}^{I}\sum_{j=1}^{J} \frac{Z_{ij}^2}{n} = \frac{\chi^2}{n} \tag{32}$$

Cette égalité est fondamentale : elle prouve que l'Inertie Totale n'est rien d'autre que la statistique du $\chi^2$ de Pearson normalisée par l'effectif total $n$. En d'autres termes, l'Inertie Totale mesure la quantité globale d'information (ou d'écart à l'indépendance) contenue dans le tableau, indépendamment de la taille de l'échantillon. (Pour ceux du fond qui n'auraient pas suivi.

**Un exemple vaut mieux que mille mots** J'espère ne pas trop vous avoir perdu. Nous allons prendre un exemple pour mieux comprendre le tout. Je vous rappel qu'on va bosser sur ce tableau-ci :

| | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Departement** | | | | | | | | | | | | | |
| **Seine-Saint-Denis** | 0.005074 | 0.021434 | 0.202739 | 0.012529 | 0.118831 | 0.051494 | 0.490898 | 0.010844 | 0.035629 | 0.032184 | 0.006744 | 0.011599 | 1.0 |
| **Val-de-Marne** | 0.004251 | 0.025386 | 0.290988 | 0.015184 | 0.118153 | 0.073728 | 0.326669 | 0.014050 | 0.054161 | 0.055210 | 0.006332 | 0.015888 | 1.0 |
| **Val-d'Oise** | 0.004517 | 0.018996 | 0.260895 | 0.017185 | 0.171986 | 0.070931 | 0.331705 | 0.011507 | 0.039106 | 0.049856 | 0.006329 | 0.016987 | 1.0 |
| **Essonne** | 0.004758 | 0.022658 | 0.276503 | 0.019515 | 0.177937 | 0.066030 | 0.281221 | 0.013353 | 0.049689 | 0.055545 | 0.007238 | 0.025554 | 1.0 |
| **Hauts-de-Seine** | 0.002976 | 0.016999 | 0.371084 | 0.014534 | 0.083656 | 0.081009 | 0.257686 | 0.013576 | 0.060798 | 0.080325 | 0.004781 | 0.012576 | 1.0 |
| **Paris** | 0.002740 | 0.016366 | 0.353363 | 0.011505 | 0.055380 | 0.081595 | 0.300809 | 0.021706 | 0.076079 | 0.065934 | 0.005433 | 0.009090 | 1.0 |
| **Seine-et-Marne** | 0.005205 | 0.019432 | 0.249963 | 0.020875 | 0.235635 | 0.072463 | 0.258680 | 0.011882 | 0.040425 | 0.055722 | 0.006960 | 0.022759 | 1.0 |
| **Yvelines** | 0.003766 | 0.015914 | 0.334094 | 0.018584 | 0.137674 | 0.087449 | 0.228898 | 0.012282 | 0.054949 | 0.083225 | 0.005381 | 0.017783 | 1.0 |
| **Average Line Profile** | 0.003992 | 0.019172 | 0.301931 | 0.015927 | 0.129735 | 0.074653 | 0.302445 | 0.014318 | 0.053991 | 0.061884 | 0.006027 | 0.015925 | 1.0 |

Rappel

Calculons la distance entre le département Seine-Saint-Denis et l'origine (le profil-moyen) :

$$d^2(\text{Seine-Saint-Denis}, LM) \approx$$
$$\frac{(0.005074 - 0.003992)^2}{0.003992}\ (\text{Arthaud}) + \frac{(0.021434 - 0.019172)^2}{0.019172}\ (\text{Roussel})$$
$$+\frac{(0.202739 - 0.301931)^2}{0.301931}\ (\text{Macron}) + \frac{(0.012529 - 0.015927)^2}{0.015927}\ (\text{Lassalle})$$

$$+\frac{(0.118831-0.129735)^2}{0.129735}\text{ (Le Pen)}+\frac{(0.051494-0.074653)^2}{0.074653}\text{ (Zemmour)}$$
$$+\frac{(0.490898-0.302445)^2}{0.302445}\text{ (Mélenchon)}+\frac{(0.010844-0.014318)^2}{0.014318}\text{ (Hidalgo)}$$
$$+\frac{(0.035629-0.053991)^2}{0.053991}\text{ (Jadot)}+\frac{(0.032184-0.061884)^2}{0.061884}\text{ (Pécresse)}$$
$$+\frac{(0.006744-0.006027)^2}{0.006027}\text{ (Poutou)}+\frac{(0.011599-0.015925)^2}{0.015925}\text{ (Dupont-Aignan)}\approx 0.18197$$

Nous devons faire de même pour chacun des départements d'Île de France.

$$P_{\text{Seine-Saint-Denis}}=\frac{543\,151}{5\,483\,974}\approx \mathbf{0,09904}$$

$$Inertie(\text{Seine-Saint-Denis})=P_{\text{Seine-Saint-Denis}}\times d^2(93,LM)$$
$$Inertie(\text{Seine-Saint-Denis})=0,09904\times 0,18197\approx \mathbf{0,01802}$$

A REFAIRE LES CALCuLS Ceci est un exemple de calcul d'inertie partielle. En faisant de même pour toutes les lignes, puis en sommant toutes les inerties obtenues, vous retomberez directement sur l'inertie totale. Vous pouvez même via Python représenter cela sous la forme d'un tableau de données : Il faut ajouter tab5, tab6 et code8

**Produit scalaire** Pour généraliser cette notion d'inertie, je trouve intéressant de vous écrire la distance du $\chi^2$ de la manière suivante, manière qui va devenir super utile pour pouvoir analyser algèbriquement cette notion :

$$\langle L_i,L_{i'}\rangle_{\chi^2}=\sum_{j=1}^{C}\frac{L_{ij}\times L_{i'j}}{LM_j}=L_i^TD_c^{-1}L_{i'}, \tag{33}$$

avec $D_c$ la matrice diagonale des $\frac{1}{LM_j}$. Bon ici, nous n'avons qu'un produit scalaire, que l'on appellera Produit Scalaire du $\chi^2$. Mais il va devenir très facile de définir comme ceci la distance entre deux profils :

$$d^2_{\chi^2}(L_i,L_{i'})=\|L_i-L_{i'}\|^2_{\chi^2}=\langle L_i-L_{i'},L_i-L_{i'}\rangle_{\chi^2}=\sum_{j=1}^{C}\frac{1}{LM_j}(L_{ij}-L_{i'j})^2 \tag{34}$$

Et voilà, je vous avais pourtant prévenu que le $\chi^2$ n'était qu'un simple problème de minimisation de norme. Avec cette habilité, vous pouvez maintenant non seulement calculer la distance d'un profil par rapport à un autre profil. Si votre $L_{i'}$ est en réalité LM, on retombe alors habilement sur nos pattes. On peut alors recomposer la notion d'inertie pour deux départements distincts :

$$\mathcal{I}_{ii'}=\sqrt{P_i\times P_{i'}}\times\sum_{j=1}^{J}\frac{(L_{ij}-LM_j)(L_{i'j}-LM_j)}{LM_j} \tag{35}$$

on remarquera que si on met le MEME DEPARTEMENT, on retrouve notre formule d'inertie partielle que l'on a définie plus tôt. Je vous laisserai aussi le soin de regarder les similitudes que l'on retrouve avec la formule de covariance habituelle.

**De la matrice d'inertie à son eigen-décomposition** Nous voilà avec une bien belle matrice d'inertie. Comme la plupart des matrices, notre but est d'étudier sa diagonalisation. Commençons par nous attarder sur la matrice des résidus standardisés de Pearson ($Z$), et regardons ce que l'on trouve en calculant $\frac{ZZ^T}{n}$ : Effectivement, cnous venons de retrouver la matrice d'inertie pour chaque département. Je ne vous l'ai toujours pas dit mais $Tr(\frac{ZZ^T}{n}) = \phi^2$, donc l'INERTIE TOTALE :). En pratique, on utilisera TOUJOURS ce calcul en AFC car il est rapide et surtout TRES UTILE puisqu'il nous donne toutes les informations que l'on veut. En plus de cela, on remarquera que la matrice est une matrice symétrique : $(ZZ^t)^t = ZZ^T$, ce qui la rend diagonalisable à coup sûr et qu'il existe forcément une base orthonormée composée de vecteurs propres de la matrice.

# Démonstration : Valeurs Propres et Inertie Totale

L'inertie totale $\mathcal{I}_{tot}$ est définie comme la somme des inerties de chaque point (département), ce qui correspond à la trace de la matrice d'inertie $W = \frac{ZZ^T}{n}$.

## 1. Définition par la Trace

L'inertie totale est la somme des éléments diagonaux de la matrice $W$ :

$$\mathcal{I}_{tot} = \mathrm{Tr}(W) = \sum_{i=1}^{I} W_{ii} \tag{36}$$

Chaque $W_{ii}$ représente l'inertie du département $i$ par rapport au centre de gravité.

## 2. Relation avec les Valeurs Propres

D'après les propriétés de l'algèbre linéaire, la trace d'une matrice est invariante par changement de base. Si $W$ est diagonalisable (ce qui est le cas car elle est symétrique et réelle), elle est semblable à une matrice diagonale $\Delta$ contenant ses valeurs propres $\lambda_k$ :

$$\mathrm{Tr}(W) = \mathrm{Tr}(\Delta) = \sum_{k=1}^{K} \lambda_k \tag{37}$$

## 3. Conclusion

On en déduit la relation fondamentale de l'Analyse des Correspondances : l'inertie totale est égale à la somme des variances capturées par chaque axe factoriel :

$$\mathcal{I}_{tot} = \sum_{i=1}^{I} \text{Inertie}(i) = \sum_{k=1}^{K} \lambda_k \tag{38}$$

Cette égalité démontre que la décomposition en valeurs propres permet de partitionner l'inertie totale du tableau sur des axes orthogonaux.

En réalité, une fois les valeurs propres obtenues, il suffit de récupérer celles qui offrent la contribution la plus puissante, puis de récupérer une base de vecteurs propres associés qui soit orthonormale (grâce au théorème spectral). De notre côté, Python s'en chargera très bien. Ce qui devient extrêmement puissant dans ce processus, c'est que l'on va pouvoir ne récupérer que deux ou trois axes (ceux avec les valeurs propres les plus fortes), et nous pourrons aussi calculer le % d'inertie associé à chacun des axes, puis représenter les points sur un graphique grâce à la formule du changement de base.

**Comment transporter nos points en les projetant ?** Commençons déjà par rappeler que notre matrice aura comme rang au plus $min(L-1, C-1)$. Si vous n'avez pas compris cela, reprenez la notion de degré de liberté du $\chi^2$ dans le détail et appliquez-la à la notion de rang d'une matrice rectangulaire. Bien sûr, ce n'est qu'une majoration, il se peut que la matrice soit d'un rang plus petit (ce qui est EXTREMEMENT RARE). Pour notre matrice d'inertie qui est dans notre cas de taille $L^2$, elle ne pourra pas avoir plus de L-1 valeurs propres non nulles. En réalité, Python, avec ses grands sabots, vous en proposera surement une car... Il fait des approximations, et les approximations font que des erreurs se glissent. Ces erreurs deviennent alors la cause de l'existence d'une $L^e$ valeur propre qui devrait être nulle, et donc ne pas contribuer à l'inertie totale. Ok, imaginons que nous avons récupéré toutes les valeurs propres ($\lambda_1 ... \lambda_k$ de la matrice d'inertie et que nous avons par ailleurs la base de vecteurs propres orthogonaux associés $(u_1, ...u_k)$ et que nous les réarrangions par indice de manière arbitraire dans l'ordre décroissant des valeurs propres. Pour l'explication, je ne garderai que les deux plus grandes valeurs propres contribuant à l'inertie (puisque c'est le principal usage qu'on fait de l'AFC aujourd'hui).

Premièrement, pour un soucis de lisibilité des graphiques, commençons par passer les deux valeurs propres à la racine carré, cela nous permettra d'avoir des échelles de distances sur les graphiques plus interprétables que des carrés. Les vecteurs propres, qui définissent l'orientation des axes factoriels, ne changent quant à eux pas. Multiplions les vecteurs propres par la racine carré des valeurs propres pour pouvoir garder visuellement les importances.
Si on ne garde que les deux premiers axes, la matrice finale $F$ (pour le plan 2D)

ressemble à ceci :

$$F = \begin{pmatrix} | & | \\ \sqrt{\lambda_1}u_1 & \sqrt{\lambda_2}u_2 \\ | & | \end{pmatrix}$$

Dans le cas de nos élections, nous retrouverons en ligne les 8 départements, et en colonne les deux axes factorielles que l'on a récupéré. Il devient alors super aisé d'afficher ceci sur un graphique : la première colonne représente la coordonnée en x (premier axe factoriel) et la seconde colonne représente la cordonnée en y (le second axe factoriel), et ainsi de suite...

Je pense que vous comprendrez maintenant pourquoi l'analyse en colonne fonctionnera de la même manière. Seulement, il faudra faire attention à calculer $Z^tZ$ (pour que la matrice soit de la forme $C \times C$ et non $L \times L$. Cela assurera d'avoir une analyse par candidat. Je vous laisserai le soin de regarder si les valeurs propres et les vecteurs propres restet les mêmes. Je ne vais pas vous refaire le cours, car il est très analogue. Néanmoins, je vous encourage à le faire par vous-même pour vous perfectionner.

**La notion d'attraction répulsion** Promis, c'est bientôt fini. La métrique que nous nous apprêtons à utiliser permet de savoir si les points sur le graphique ont tendance à se "fuire" (le désintérêt total) ou alors si elles ont tendance à s'attirer (l'intérêt total).

Pour un département $i$ et un candidat $j$, le coefficient est :

$$C_{ij} = \frac{f_{ij}}{f_{i.} \times f_{.j}}$$

Où :

$f_{ij}$ : Proportion réelle de votes pour le candidat $j$ dans le département $i$.

$f_{i.}$ : Poids total du département (sa masse).

$f_{.j}$ : Poids total du candidat (son score régional moyen).

Formule intuitive (Profils) : $C_{ij} = \frac{\text{Profil ligne } i \text{ sur la modalité } j}{\text{Profil moyen de la modalité } j}$

**Lecture de cette métrique** Si $C_{ij} > 1$, il y a une relation d'attraction. Le département vote plus pour ce candidat que la moyenne. Si $C_{ij} = 1$, il y a une relation d'indépendance stricte. Le département vote exactement comme la moyenne régionale. (Nous avons donc réussi à résoudre le problème que je vous avait dit dan sle préambule de cette partie.) Finalement, si $C_{ij} < 1$, il y a une relation de répulsion. Le département vote moins pour ce candidat que la moyenne.

**Graphique simultanés** Je vous ai dit qu'on peut faire indépendant l'analyse des lignes et des colonnes, et c'est vrai. Les valeurs propres sont identiques, mais les vecteurs propres changent. Cela est normal, puisque les coordonnées des points dans le nouveau repère orthonormé pour les départements n'ont aucune raison d'être les mêmes que ceux des candidats. Vous pouvez donc construire ce qu'on appelle un Biplot, c'est–à-dire un graphique qui affiche simultanément les candidats et les départements. Néanmoins, faîtes attention à comment vous interprétez vos points :

- Distance entre deux départements : Si deux départements sont proches, ils ont des profils de vote similaires.
- Distance entre deux candidats : S'ils sont proches, ils sont votés par les mêmes types de départements.
- Distance entre un département et un candidat : C'est là que l'attraction/répulsion intervient. Si le département 93 est proche de Mélenchon, c'est qu'il y a une forte attraction des électeurs du 93 pour Mélenchon (le coefficient $C_{ij} > 1$).

**Résumé de l'AFC** Il est grand temps de vous écrire de manière synthétique la recette de cuisine de l'AFC, vous la méritez amplement mes petits chefs cuistots prodiges !

Voici le résumé complet :

- Commencer par faire une analyse descriptive du tableau de contingence pour commencer à voir quelles sont les tendances globales dans votre jeu de données.
- Définir sur quoi veut se porter notre analyse ? Les colonnes ou les lignes ? ? ?
- Calculer en conséquence la matrice d'inertie.
- Calculer ses valeurs propres.
- Récupérer ses vecteurs propres.
- Récupérer seulement les axes factoriels qui nous intéressent.
- Reconstruire la nouvelle matrice de dimension réduite
- Dans le cas d'une matrice à deux ou trois colonnes, faîtes un graphique pour rendre l'analyse visuelle.
- Utilisez tous les indicateurs et la théorie autour du $\chi^2$ que vous avez apprise dans ce cours.

# 3 Une mise en pratique complète de l'AFC dans l'analyse des élections présidentielles 2022

Avant même de débuter, je vous invite à regarder ce tutoriel-ci fait par le créateur du package lui même (je n'ai pas trouvé d'autres documentations).

Je sais que nos analyses ont pu être un peu fouillies puisqu'elles ont été discontinues lors de la lecture. Le but decette section est de pallier à ce problème

en vous montrant comment faire de A à Z une AFC en utilisant un package Python prévu à cet effet. J'utiliserai pour ma part le package **fanalysis** pour l'AFC ainsi que **pingouin** (qui utilise scipy et statsmodel) pour l'analyse du $\chi^2$ et le calcul de ses métriques avancées. Notez que scikitlearn propose aussi ce genre de choses, mais elle est uniquement orienté machine learning et non pas exploration des données. Il sera alors impossible de faire facilement des graphiques ni même récupérer certaines métriques indispensables.

## 3.1 Une analyse rapide du tableau de contingence

Comme je vous l'ai déjà explicité plus tôt, la première étape est de faire une rapide inspection en utilisant les statistiques descriptives. Je vous rappelle que le tableau de contingence est le suivant :

| | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Departement** | | | | | | | | | | | | | |
| **Seine-Saint-Denis** | 2756 | 11642 | 110118 | 6805 | 64543 | 27969 | 266632 | 5890 | 19352 | 17481 | 3663 | 6300 | 543151 |
| **Val-de-Marne** | 2504 | 14954 | 171409 | 8944 | 69599 | 43430 | 192427 | 8276 | 31904 | 32522 | 3730 | 9359 | 589058 |
| **Val-d'Oise** | 2392 | 10060 | 138166 | 9101 | 91081 | 37564 | 175666 | 6094 | 20710 | 26403 | 3352 | 8996 | 529585 |
| **Essonne** | 2831 | 13480 | 164503 | 11610 | 105862 | 39284 | 167310 | 7944 | 29562 | 33046 | 4306 | 15203 | 594941 |
| **Hauts-de-Seine** | 2306 | 13170 | 287494 | 11260 | 64812 | 62761 | 199640 | 10518 | 47103 | 62231 | 3704 | 9743 | 774742 |
| **Paris** | 2891 | 17267 | 372820 | 12139 | 58429 | 86088 | 317372 | 22901 | 80268 | 69564 | 5732 | 9591 | 1055062 |
| **Seine-et-Marne** | 3440 | 12843 | 165208 | 13797 | 155738 | 47893 | 170969 | 7853 | 26718 | 36828 | 4600 | 15042 | 660929 |
| **Yvelines** | 2774 | 11721 | 246062 | 13687 | 101398 | 64407 | 168585 | 9046 | 40470 | 61296 | 3963 | 13097 | 736506 |
| **Total** | 21894 | 105137 | 1655780 | 87343 | 711462 | 409396 | 1658601 | 78522 | 296087 | 339371 | 33050 | 87331 | 5483974 |

Rappel

Si on se fie aux candidats, nous remarquerons qu'il y a une surreprésentation de votes pour Macron ainsi que Mélenchon. Cela signifie donc que ces deux candidats vont beaucoup influencer dans la manière dont va se produire l'AFC. Naturellement, ces deux candidats auront un profil beaucoup plus proche de celui du profil moyen. Du côté des départements, c'est Paris qui comptabilise le plus de votes. Naturellement, Paris se retrouvera aussi plus au centre.

Lorsqu'on analyse cette fois-ci les fréquences, certains comportements d'attraction-répulsion peuvent être dors et déjà relevés. Emmanuel Macron et Paris devraient plus ou moins être proche sur le graphique final. Macron représente $\frac{6.80}{19.24} \approx 35.34\%$ des votes totaux à Paris, par exemple. Le coefficient d'attraction est donc $C_{\text{Paris, Macron}} = \frac{35,34}{30,19} \approx 1,17$. Je calculerai les tableaux contenant tous les coefficients à partir du tableau des profils, puisque je vous avais dit que c'était une autre manière de calculer ce coefficient.

**Tableau des profils** Je vous rapelle qu'ils se présentent de la manière suivante :

**Tableau des profils** Je vous rapelle qu'ils se présentent de la manière suivante :

| Departement | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 0.05 | 0.21 | 2.01 | 0.12 | 1.18 | 0.51 | 4.86 | 0.11 | 0.35 | 0.32 | 0.07 | 0.11 | 9.90 |
| Val-de-Marne | 0.05 | 0.27 | 3.13 | 0.16 | 1.27 | 0.79 | 3.51 | 0.15 | 0.58 | 0.59 | 0.07 | 0.17 | 10.74 |
| Val-d'Oise | 0.04 | 0.18 | 2.52 | 0.17 | 1.66 | 0.68 | 3.20 | 0.11 | 0.38 | 0.48 | 0.06 | 0.16 | 9.66 |
| Essonne | 0.05 | 0.25 | 3.00 | 0.21 | 1.93 | 0.72 | 3.05 | 0.14 | 0.54 | 0.60 | 0.08 | 0.28 | 10.85 |
| Hauts-de-Seine | 0.04 | 0.24 | 5.24 | 0.21 | 1.18 | 1.14 | 3.64 | 0.19 | 0.86 | 1.13 | 0.07 | 0.18 | 14.13 |
| Paris | 0.05 | 0.31 | 6.80 | 0.22 | 1.07 | 1.57 | 5.79 | 0.42 | 1.46 | 1.27 | 0.10 | 0.17 | 19.24 |
| Seine-et-Marne | 0.06 | 0.23 | 3.01 | 0.25 | 2.84 | 0.87 | 3.12 | 0.14 | 0.49 | 0.67 | 0.08 | 0.27 | 12.05 |
| Yvelines | 0.05 | 0.21 | 4.49 | 0.25 | 1.85 | 1.17 | 3.07 | 0.16 | 0.74 | 1.12 | 0.07 | 0.24 | 13.43 |
| Total | 0.40 | 1.92 | 30.19 | 1.59 | 12.97 | 7.47 | 30.24 | 1.43 | 5.40 | 6.19 | 0.60 | 1.59 | 100.00 |

FIGURE 12 – Tableau des fréquences associées

| Departement | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 0.005074 | 0.021434 | 0.202739 | 0.012529 | 0.118831 | 0.051494 | 0.490898 | 0.010844 | 0.035629 | 0.032184 | 0.006744 | 0.011599 | 1.0 |
| Val-de-Marne | 0.004251 | 0.025386 | 0.290988 | 0.015184 | 0.118153 | 0.073728 | 0.326669 | 0.014050 | 0.054161 | 0.055210 | 0.006332 | 0.015888 | 1.0 |
| Val-d'Oise | 0.004517 | 0.018996 | 0.260895 | 0.017185 | 0.171986 | 0.070931 | 0.331705 | 0.011507 | 0.039106 | 0.049856 | 0.006329 | 0.016987 | 1.0 |
| Essonne | 0.004758 | 0.022658 | 0.276503 | 0.019515 | 0.177937 | 0.066030 | 0.281221 | 0.013353 | 0.049689 | 0.055545 | 0.007238 | 0.025554 | 1.0 |
| Hauts-de-Seine | 0.002976 | 0.016999 | 0.371084 | 0.014534 | 0.083656 | 0.081009 | 0.257686 | 0.013576 | 0.060798 | 0.080325 | 0.004781 | 0.012576 | 1.0 |
| Paris | 0.002740 | 0.016366 | 0.353363 | 0.011505 | 0.055380 | 0.081595 | 0.300809 | 0.021706 | 0.076079 | 0.065934 | 0.005433 | 0.009090 | 1.0 |
| Seine-et-Marne | 0.005205 | 0.019432 | 0.249963 | 0.020875 | 0.235635 | 0.072463 | 0.258680 | 0.011882 | 0.040425 | 0.055722 | 0.006960 | 0.022759 | 1.0 |
| Yvelines | 0.003766 | 0.015914 | 0.334094 | 0.018584 | 0.137674 | 0.087449 | 0.228898 | 0.012282 | 0.054949 | 0.083225 | 0.005381 | 0.017783 | 1.0 |
| Average Line Profile | 0.003992 | 0.019172 | 0.301931 | 0.015927 | 0.129735 | 0.074653 | 0.302445 | 0.014318 | 0.053991 | 0.061884 | 0.006027 | 0.015925 | 1.0 |

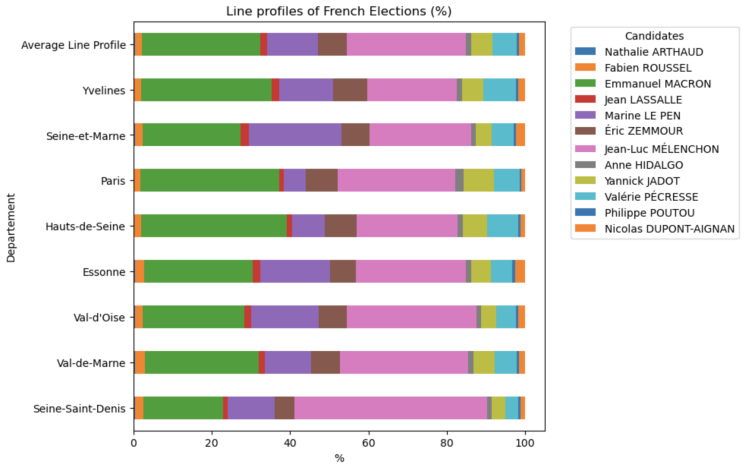
Rappel du tableau des profil-lignes

Bon, le problème de ces tableaux, c'est qu'ils ont du mal à être interprétables facilement. Pour ce faire, on avait déjà fait deux beaux graphiques :

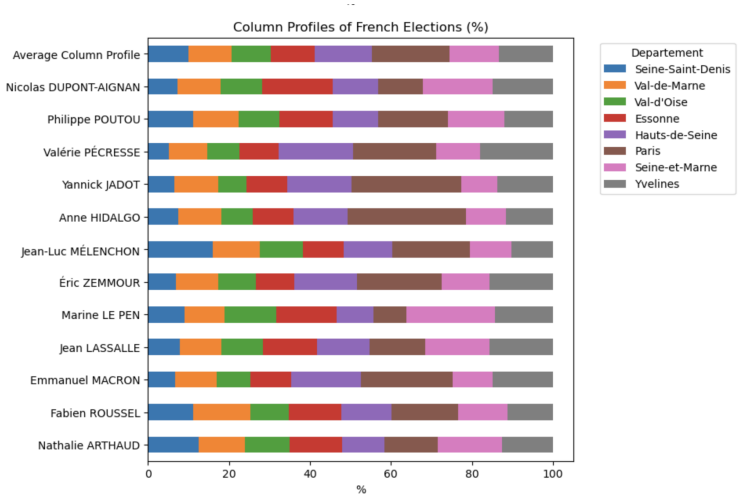
| Departement | Nathalie ARTHAUD | Fabien ROUSSEL | Emmanuel MACRON | Jean LASSALLE | Marine LE PEN | Éric ZEMMOUR | Jean-Luc MÉLENCHON | Anne HIDALGO | Yannick JADOT | Valérie PÉCRESSE | Philippe POUTOU | Nicolas DUPONT-AIGNAN | Average Column Profile |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Seine-Saint-Denis | 0.125879 | 0.110732 | 0.066505 | 0.077911 | 0.090719 | 0.068318 | 0.160757 | 0.075011 | 0.065359 | 0.051510 | 0.110832 | 0.072139 | 0.099043 |
| Val-de-Marne | 0.114369 | 0.142233 | 0.103522 | 0.102401 | 0.097825 | 0.106083 | 0.116018 | 0.105397 | 0.107752 | 0.095830 | 0.112859 | 0.107167 | 0.107414 |
| Val-d'Oise | 0.109254 | 0.095685 | 0.083445 | 0.104198 | 0.128019 | 0.091755 | 0.105912 | 0.077609 | 0.069946 | 0.077800 | 0.101422 | 0.103010 | 0.096570 |
| Essonne | 0.129305 | 0.128214 | 0.099351 | 0.132924 | 0.148795 | 0.095956 | 0.100874 | 0.101169 | 0.099842 | 0.097374 | 0.130287 | 0.174085 | 0.108487 |
| Hauts-de-Seine | 0.105326 | 0.125265 | 0.173631 | 0.128917 | 0.091097 | 0.153301 | 0.120367 | 0.133950 | 0.159085 | 0.183372 | 0.112073 | 0.111564 | 0.141274 |
| Paris | 0.132045 | 0.164233 | 0.225163 | 0.138981 | 0.082125 | 0.210281 | 0.191349 | 0.291651 | 0.271096 | 0.204979 | 0.173434 | 0.109824 | 0.192390 |
| Seine-et-Marne | 0.157121 | 0.122155 | 0.099777 | 0.157963 | 0.218899 | 0.116985 | 0.103080 | 0.100010 | 0.090237 | 0.108518 | 0.139183 | 0.172241 | 0.120520 |
| Yvelines | 0.126701 | 0.111483 | 0.148608 | 0.156704 | 0.142521 | 0.157322 | 0.101643 | 0.115203 | 0.136683 | 0.180616 | 0.119909 | 0.149970 | 0.134302 |
| Total | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 |

Rappel du tableau des profil-lignes

Ces graphiques nous permettent de déceler à l'oeil qu'il y a attraction entre Mélenchon et la Seine-Saint-Denis. Au contraire, il semblerait y avoir répulsion entre Marine Le Pen et Paris. J'avoue que ces graphiques sont très utiles, mais ils sont assez long à analyser et ne permettent pas de quantifier les écarts. Faisons alors une heatmap qui calcule tous les coefficients d'attraction répulsion pour chaque candidat ET chaque département d'Île-de-France :




Rappel de l'histogramme des profil-lignes




Rappel de l'histogramme des profil-colonnes

Qui l'eût cru qu'Hidalgo serait la candidate favorite des parisiens ? (1.52) Les parisiens sont par ailleurs anti RN à souhait (0.43). Mélenchon est un dieu en Seine-Saint-Denis (1.62), mais gare aux racistes de Seine-et-Marne qui n'attendent que le grand remplacement ! (1.82) Plus sérieusement, essayons d'imaginer les blocs que pourraient former nos données sous AFC.

```python
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Calcul des coefficients (ton calcul)
# On s'assure de diviser le profil ligne par le profil moyen
coeff_attraction_repulsion = line_profile_cont / line_profile.drop(columns="Total").loc["Average Line Profile"]

# 2. Configuration du style
plt.figure(figsize=(13, 7))
sns.set_theme(style="white") # Fond propre

# 3. Création de la Heatmap
# 'RdYlGn' : Rouge pour répulsion (<1), Jaune pour neutre (1), Vert pour attraction (>1)
# 'center=1' : Définit le pivot de la couleur sur l'indépendance statistique
ax = sns.heatmap(coeff_attraction_repulsion,
                 annot=True,
                 fmt=".2f",
                 cmap="RdYlGn",
                 center=1,
                 linewidths=1,
                 linecolor='white',
                 cbar_kws={'label': 'Indice de liaison (Attraction > 1)'})

# 4. Personnalisation des axes et titres
plt.title("Analyse des Liaisons : Attractions et Répulsions", fontsize=15, pad=20)
plt.xlabel("Candidats", fontsize=12, labelpad=10)
plt.ylabel("Départements", fontsize=12, labelpad=10)
plt.xticks(rotation=45)

# 5. Affichage
plt.tight_layout()
plt.show()
```

FIGURE 13 – Le code Python

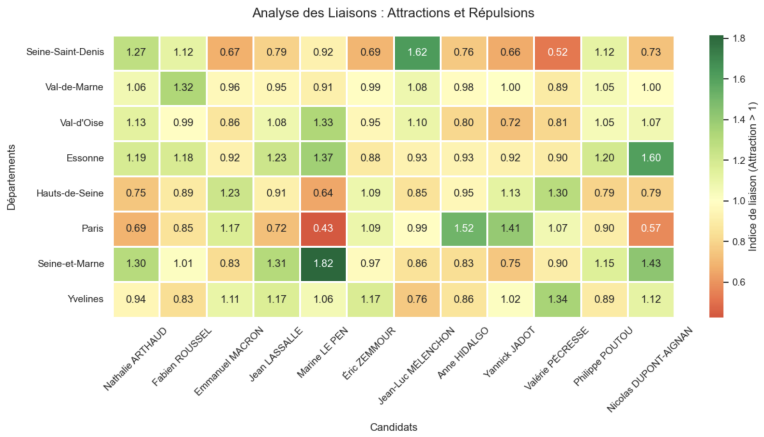



FIGURE 14 – La heatmap associée

# Analyse des Liaisons : Trois Tendances Majeures

L'examen de la matrice des indices de liaison (Heatmap d'attraction-répulsion) permet d'identifier trois structures géographiques et politiques fortes qui organiseront l'espace factoriel :

### 1. L'axe de la Droite Libérale et Conservatrice (Yvelines / Hauts-de-Seine)

Il existe une corrélation spatiale très nette entre les départements socialement favorisés et les candidats de la droite traditionnelle :

- **Valérie Pécresse** enregistre ses indices d'attraction les plus élevés dans les **Yvelines (1,34)** et les **Hauts-de-Seine (1,30)**.
- **Éric Zemmour** suit une dynamique proche avec une attraction marquée dans les **Hauts-de-Seine (1,09)** et à **Paris (1,09)**.
- À l'inverse, ces candidats subissent une **répulsion systématique** en Seine-Saint-Denis (indices respectifs de 0,52 et 0,69).

### 2. La Polarisation de la Grande Couronne (Seine-et-Marne / Essonne)

Tandis que le centre de l'agglomération (Paris) rejette massivement les votes souverainistes, la périphérie exerce une force d'attraction inverse :

- **Marine Le Pen** présente son indice d'attraction maximal en **Seine-et-Marne (1,82)**, contrastant violemment avec sa répulsion à Paris (0,43).
- **Nicolas Dupont-Aignan** s'inscrit dans cette même logique avec une attraction forte en **Seine-et-Marne (1,43)** et en **Essonne (1,60)**.
- **Jean Lassalle** trouve également son ancrage principal dans ces zones moins denses (1,31 en Seine-et-Marne).

### 3. La Neutralité Relative du Val-de-Marne

Le département du **Val-de-Marne** se distingue par des indices de liaison gravitant presque tous autour de **1,00** (ex : 1,08 pour Mélenchon, 0,96 pour Macron, 1,00 pour Dupont-Aignan).

- **Interprétation :** Ce département présente le profil de vote le plus proche du **profil moyen régional**.
- **Impact graphique :** Dans l'Analyse des Correspondances (AFC), ce point sera situé à proximité immédiate de l'origine des axes, car il ne contribue que faiblement à la dispersion (inertie) du nuage de points.

**Transition** Nous avons fini notre analyse "rapide" du tableau de contingence. Il est grand temps de faire l'AFC et d'analyser les résultats. J'insistais vraiment sur l'analyse en amont puisqu'elle est tout aussi importante que le résultat que puisse amener votre AFC. L'AFC ne doit être qu'un outil de vérification, voire de découverte des liens, pas un messie qui a raison sur tout.

## 3.2 La réduction de dimension par l'AFC

Comme l'AFC est très utilisée à la fois pour éviter le fléau de la dimension, mais aussi et surtout, pour pouvoir représenter graphiquement votre jeu de données, je vous propose qu'on fasse un rapide tour de ces deux techniques.

### 3.2.1 De l'AFC à la représentation graphique

Pour représenter graphiquement l'AFC, vous devrez installer le package **fanalysis** dans votre environnement de code. Si ceci est fait, écrivez le code suivant :

```
from fanalysis.ca import CA
import pandas as pd

# --- PARTIE AFC avec fanalysis ---
afc = CA(row_labels=X.index.values, col_labels=X.columns.values)
afc.fit(X.values)

# Affiche le pourcentage d'inertie par axe
afc.plot_eigenvalues()

# Génère le plan factoriel (Axe 1 et Axe 2 par défaut)
afc.mapping(num_x_axis=1, num_y_axis=2, figsize=(12, 10))

# Contribution des départements à l'Axe 1
afc.plot_row_contrib(num_axis=1)

# Contribution des candidats à l'Axe 1
afc.plot_col_contrib(num_axis=1)
```

Rappel du tableau des profil-lignes

**Précisions** Ici, pour afficher notre graphique, nous avons utilisé la méthode **mapping**. Les variables locales **num_x_axis** et **num_y_axis** représentent respectivement le numéro de l'axe factoriel (rangé par ordre décroissant d'importance en termes d'inertie) que vous voulez afficher respectivement en abscisse et en ordonnée. Il ne me semble pas qu'il soit possible de faire néanmoins de graphiques en 3D. Si vous voulez faire ceci, récupérez les coordonnées lignes avec l'attribut **row_coord** et les coordonnées des points colonnes **col_coord**, puis générez votre visualisation en 3D à l'aide de matplotlib ou plotly. De même, si vous n'avez que faire des points lignes ou colonnes dans votre visualisation, il existe la méthode **mapping_row** (respectivement mapping_col) qui vous permet d'obtenir seulement les points lignes ou colonnes dans votre visualisation. Cela permet notamment de pas saturer votre graphique d'information. De même, je vous conseille d'utiliser des **figsize** assez grandes puisque sinon, le nom des points pourraient avoir tendance à se superposer. Attention néanmoins, ne vous faîtes pas avoir par la taille du graphique. Plus le graphique est grand, plus les distances affichées sont grandes... Mais intrinsèquement, vous avez la même distance de $\chi^2$ puisque les mesures des axes s'adaptent en conséquence. Je vous

laisserai le soin de faire vos propres expériences sur le mapping uniquement des lignes et des colonnes, puis de vous assurer par la même occassion que le mapping total n'est qu'une superposition des deux.

Si vous avez compris toutes ces subtilités, voici le graphique que nous obtenons en résultat :

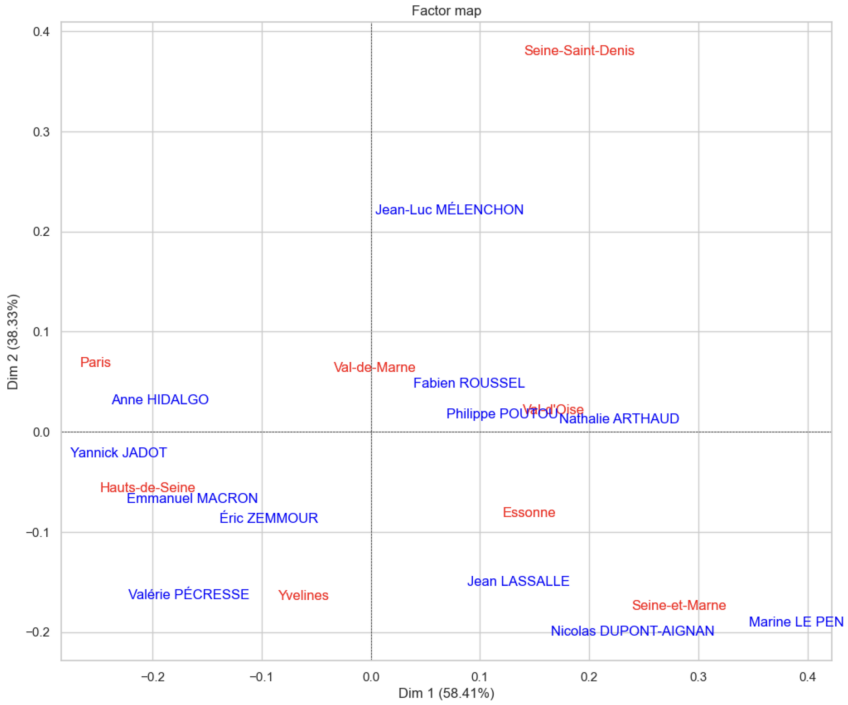



FIGURE 15 – Visualisation 2D des résultats de l'ACP

De mon côté, pour un soucis de visibilité sur le support papier, j'ai du faire des concessions quant à la superposition des noms des points. Par la même occasion, cela illustre le point que je vous disais plus tôt : Poutou, Arthaud et le Val d'Oise se superposent, ce qui rend difficile la lecture. Hormis cela, commençons déjà par analyser le nom des axes. Vous remarquerez que fanalysis fait apparaître le pourcentage d'inertie totale capturée par l'axe factoriel en question :dim $1 = 58.41\%$ et dim $2 = 38.33\%$, ce qui correspond à une explication de $58.41\% + 38.33\% = 96.74\%$ de l'inertie totale. Oui je sais, c'est beaucoup dans notre cas, donc notre représentation graphique sera très précise et pertinente pour ce cas-ci. Attention il existe des jeux de données (je pense notamment à ceux où il y a une centaine de variables différentes) où nous devons avoir beaucoup plus que deux axes factoriels principaux pour pouvoir capter un maximum d'inertie. Comme avec nos coefficients de répulsion, on remarquera que Paris a une forte attractiion envers Hidalgo et Jadot, là où la répulsion envers Marine

Le Pen se fait fortement ressentir (les points sont très distants). Au contraire, Le Pen se retrouve sans problème proche de la Seine-et-Marne, ce qui correspondait plus ou moins à nos attentes. Les points les plus au centre (0,0) du graphique sont ceux qui se rapprochent le plus de la moyenne théorique sous couvert d'indépendance. Dans notre cas, Roussel serait, avec Poutou, l'unique candidat dont les coefficients d'attraction-répulsion au sein de chacun des départements serait proche de 1. (cf Heatmap plus haut). Je veux aussi vous faire parvenirque ce n'est pas parce que des points sont très proches que les électeurs votent indifféremment Poutou ou Arthaud par exemple. Bien au contraire ! Il y a forcément quelque chose qui déclenche le choix des électeurs, une information qui ne semble pas être récoltée par les axes factoriels $n^{os}$ 1 et 2, mais qui le serait probablement en 3D si vous choisissiez l'axe factoriel adéquat pour la hauteur. L'AFC ici ne nous permet que de conclure quant à une structure de vote similaire.

**Note épistémologique** Bon nous pourrions analyser ces graphiques pendant des heures, et je vous engage à le faire, puis de créer un rapport très détaillé de vos analyses, en correspondance avec la réalité politique du terrain. Ceci sort bien évidemment du cadre purement pédagogique de mon article, et c'est pourquoi je ne le ferai pas. Simplement, je tiens à vous informer quant à la prudence épistémologique que vous devez avoir face à un tel graphique. C'est pour cela que nous allons analyser deux métriques importantes par la suite : la contribution d'un point à un axe factoriel et le $cos^2$ de l'angle formé par le point et l'axe factoriel. La contribution d'un point à la formation d'un axe permet de montrer quels sont les points qui ont été les plus influents dans la construction d'un axe. De notre coté, pour l'axe factoriel $n^o$ 1, je vous fais le pari non risqué que ce soit Paris (pour les points lignes) et que ce soit Le Pen (pour les points colonnes) qui aient les plus grandes contributions. A l'inverse, ce serait peut être Mélenchon et Seine-Saint-Denis qui auront un % de contribution assez élevé pour l'axe factoriel $n^o$ 2. (ici je pense à Mélenchon car il a raflé énormément de voix en Île-de-France, donc il pèse forcément plus dans le déplacement des axes factoriels. De plus, mélenchon est très excentré, donc les deux arguments mis bout à bout nous permettent de faire cette hypothèse. ) La contribution aux axes (que l'on va par la suite déterminer mathématiquement via une méthode de **fanalysis**), nous permettent notamment de savoir comment les axes factoriels se sont formés, et donc de pouvoir donner plus aisément un SENS à ces axes. D'autre part, le $cos^2$ pour faire très sommaire, permet de déterminer la précision d'interprétation du point en fonction d el'axe factoriel considéré. Plus le $cos^2$ d'un point par rapport à un axe factoriel est élevé, plus cet axe permet d'expliquer au mieux les comportements électoraux vis-à-vis dudit point. En bref, si votre $cos^2$ est très faible, c'est-à-dire en dessous de 0.20, n'essayez pas d'interpréter le point en fonction d'un axe puisqu'il y a de grandes chances que sa position dans le graphique ne soit qu'un artefact (puisqu'il faut, même si cela n'a pas beaucoup de sens, pouvoir afficher un point en fonction de ses coordonnées). Si votre $cos^2$ est autour des 0.50, vous pouvez être serein quant à l'interprétabilité

des données du point quant à l'axe factoriel considéré. Si votre $cos^2$ est proche de 1, cela signifie que l'axe explique très bien votre point. Néanmoins, les autres axes ne l'expliqueront alors que partiellement (puisque la somme des $cos^2$ pour chacun des axes factoriels doivent se sommer à 1). De manière générale, si vous sommez vos $cos^2$ récupéréspour chaque axe factoriel considéré pour un point donné, vous obtiendrez la qualité globale de représentation de ce point. Aussi, garderez-vous en tête qu'un point peut avoir un $cos^2$ (angle formé par le point et l'axe factoriel au carré) tout en ayant une distance de $\chi^2$ à l'origine très petite. Cela signifie que même si nous savons sur quel axe projeter notre analyse, il n'apporte qu'une très petite dose d'informations par rapport à celles du profil moyen. Finalement, dans notre cas, l'inertie totale reconstituée est proche des 100%, donc la somme des $cos^2$ sera mathématiquement proche de 1 pour chacun des points.

J'insiste vraiment sur ces deux dernières métriques puisqu'elles sont indispensables si l'on veut faire preuve de prudence épistémologique, et donc de ne pas conclure des énormités. Voyons donc maintenant comment mettre en pratiques nos nouveaux skills via Python.

**Implémentation des contributions et cos2** Dans le package **fanalysis**, vous avez directement la méthode **plot_row_contrib** (resp. **plot_col_contrib**). De même, il y a **plot_row_cos2** (resp. **plot_col_cos2**) Cela permet de faire des diagrammes en barre. Ces graphiques sont plutôt utiles pour une première approche, mais sont loin d'être très visuelles et explicatives de la meilleure des manières. Il existe aussi une autre méthode incroyable qui va nous permettre de récupérer toutes les données que nous souhaitons. La méthode **row_topandas** (resp. **col_topandas**) permet d'afficher dans un dataframe pandas l'ensemble des coordonnées, des contributions et des $cos^2$ pour chaque axe factoriels. C'est une fonction très utile notamment pour ensuite les exporter vers Excel ou tout autre logiciel de traitement des données. Mais le problème est que cette fonction vous donne trop d'informations. Heureusement, les attributs **row_coord_** (resp. **col_coord_**), **row_contrib_** (resp. col_contrib_) et **row_cos2** (resp. **col_cos2**) nous permettent de récupérer sous forme d'array numpy uniquement la contribution, le cos2 ou encore les coordonnées des points par rapport aux axes factoriels. Bien sur, chacun des axes factoriels est rangé par ordre décroissant (de la plus grande valeur propre à la plus petite ). Avec toutes ces méthodes, nous allons pouvoir générer des heatmaps qui je trouve, seront plus facilement interprétable que les plots natif de fanalysis. (Vous commencez peut être à comprendre mon AMOUR pour les belles heatmaps.)

Bon, commençons par afficher les contributions pour les lignes et les colonnes avec les fonctions déjà implémentées dans fanalysis pour le premier axe (dans la fonction de plot, utilisez l'argument **num_axis = 1**) :

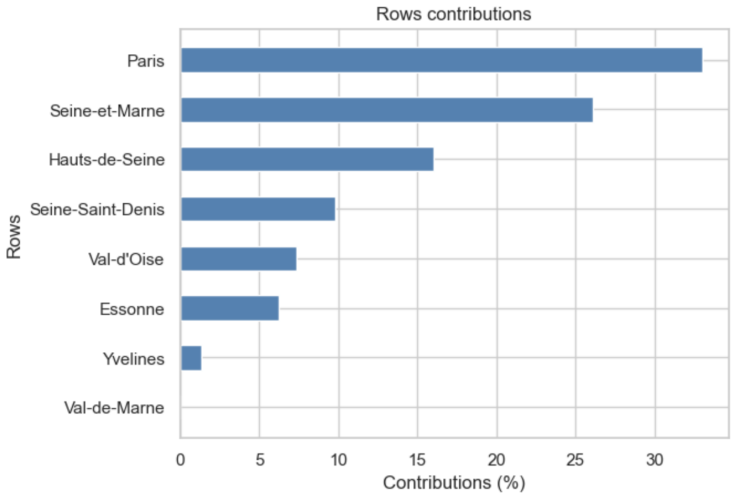



FIGURE 16 – Contributions lignes pour l'AFP pour l'axe $n^o 1$

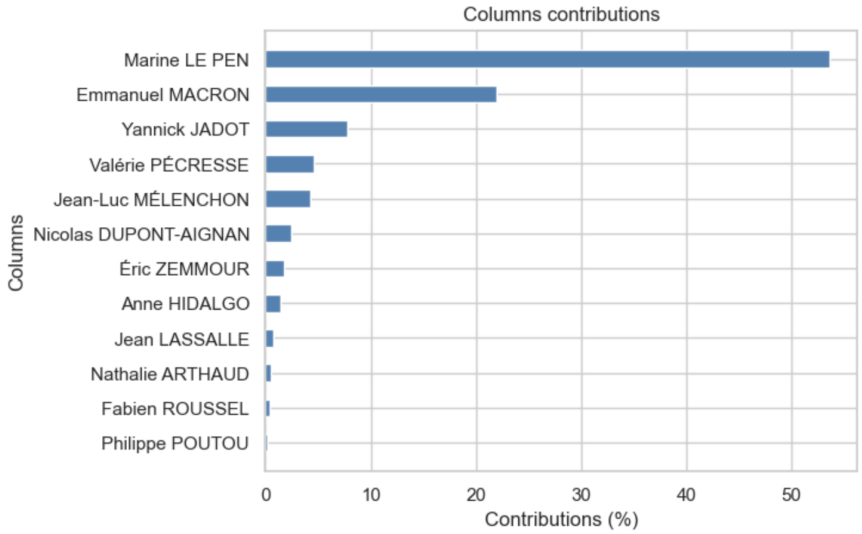



FIGURE 17 – Contribution colonnes pour l'AFP pour l'axe $n^o 1$

Comme vous pouvez le voir, nos hypothèses se confirment. C'est bel et bien Paris et Marine Le Pen qui ont les contributions les plus fortes. Cela signifie surtout que le premier axe a été créé en partie afin de faire ressortir cette répulsion très forte et prononcée entre Le Pen et Paris. Ci-dessous se trouvent les contributions lignes et colonnes liées au second facteur :

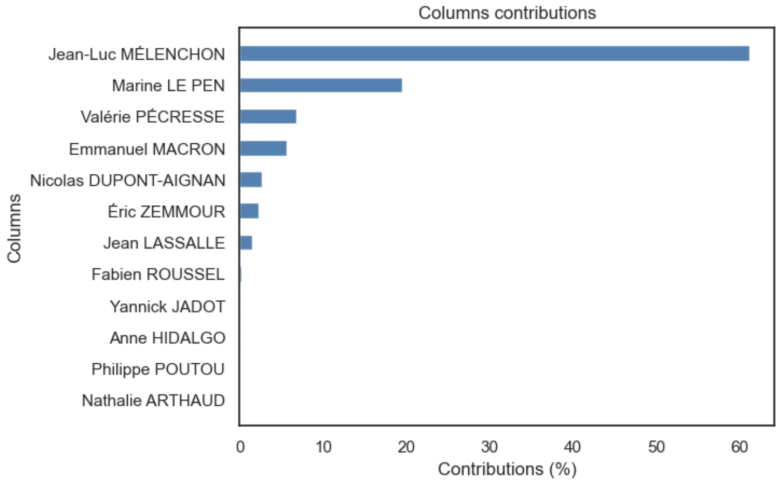



FIGURE 18 – Contributions lignes pour l'AFP pour l'axe $n^o 2$

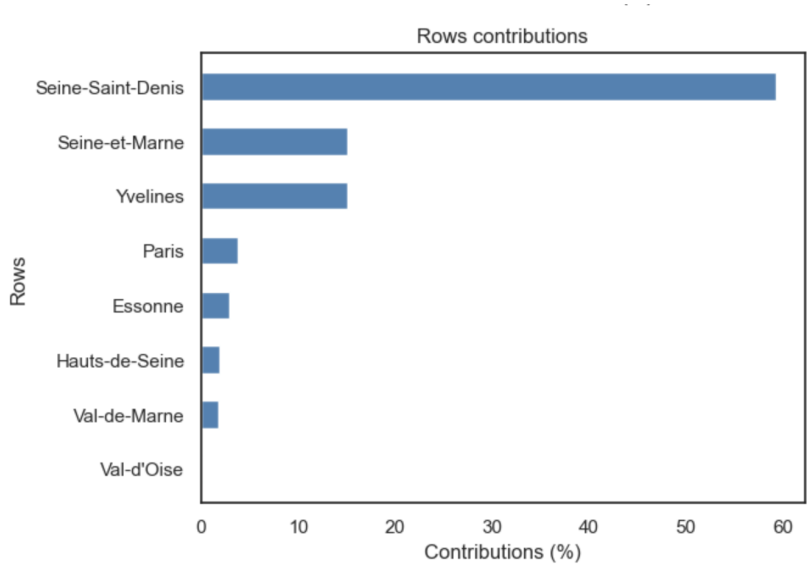



FIGURE 19 – Contribution colonnes pour l'AFP pour l'axe $n^o 2$

Effectivement, on remarquera que le second axe reçoit une contribution excessive de la Seine-Saint-Denis et Mélenchon et qui se retrouvent en répulsion totale avec Lepen, Pécresse et Macron (qui ont des contributions aussi assez fortes au graphique). Cela coïncide plutôt bien avec la disposition des points sur le graphique de l'AFC puisqu'ils sont beaucoup plus aux extrémités haut et bas.

Pour pouvoir analyser plus en détail la possibilité ou non de pouvoir analyser certains points en fonction de certains axes factoriels, nous devons calculer pour rappel le $cos^2$. Ici, voici les résultats renvoyés par Python en utilisant **plot_row_cos2(num_axis = 1)** et son homologue colonne :

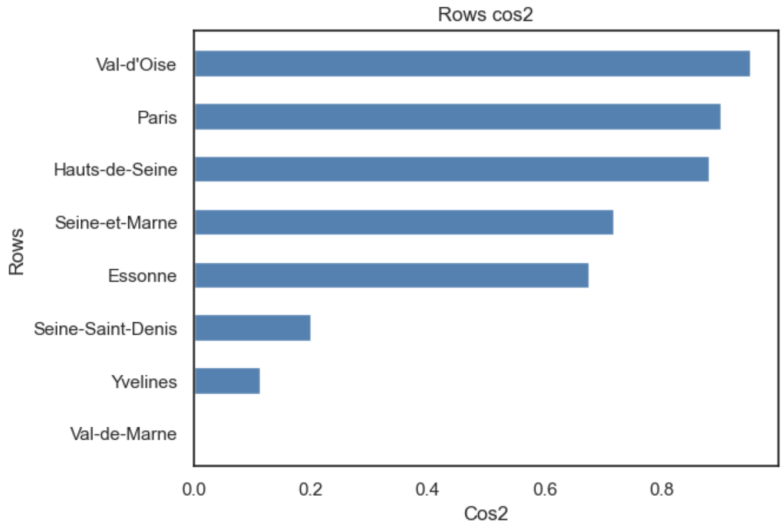



FIGURE 20 – $cos^2$ des lignes pour l'AFP sur l'axe $n^o1$

Pour l'analyse que l'on peut faire en colonne, on priviligiera donc pour l'Axe 1 le Val-d'Oise jusqu'à l'Essone.

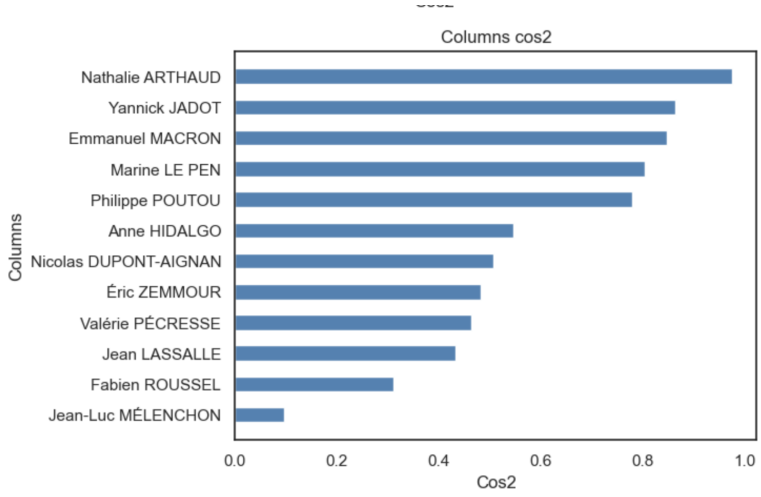



FIGURE 21 – $cos^2$ des colonnes pour l'AFP sur l'axe $n^o1$

Pour les candidats, on priviligiera Arthaud jusqu'à Pécresse et Jean Lassalle. On remarquera que cette mesure ne ment pas quant à l'inutilité d'analyser Mélenchon le long du premier axe. De ces graphiques, vous pouvez déjà deviner les résultats des seconds, mais pour un soucis d'exhaustivité, je vous les affiche aussi :

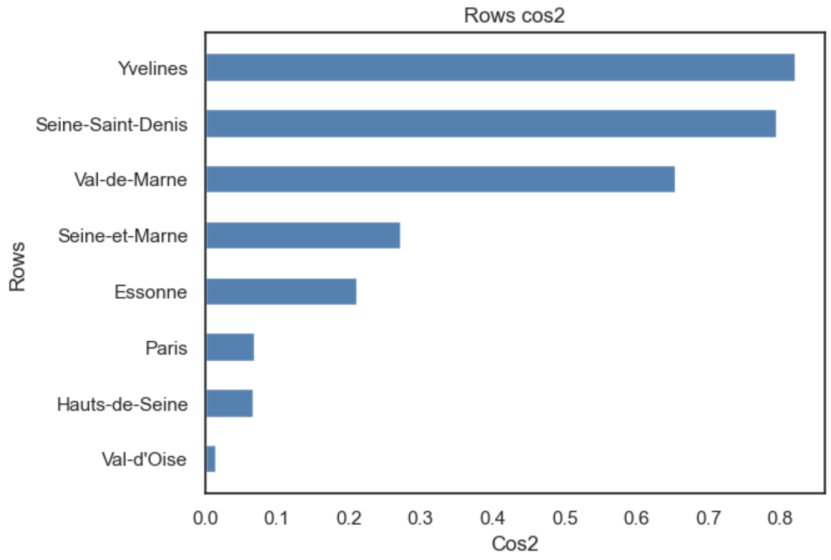



FIGURE 22 – $cos^2$ des lignes pour l'AFP sur l'axe $n^o2$

Remarquons qu'on peut donc analyser les Yvelines et la Seine-Saint-Denis.

Seulement, sur le graphique, nous pouvions voir que les Yvelines et la Seine-Saint-Denis étaient très éloignées. Il ne fait plus de doute que c'était l'un des leitmotiv de la création de l'axe factoriel $n^o$ si l'on s'en réfère de plus au % de contribution des Yvelines et de la Seine-Saint-Denis sur la création de cet axe factoriel.

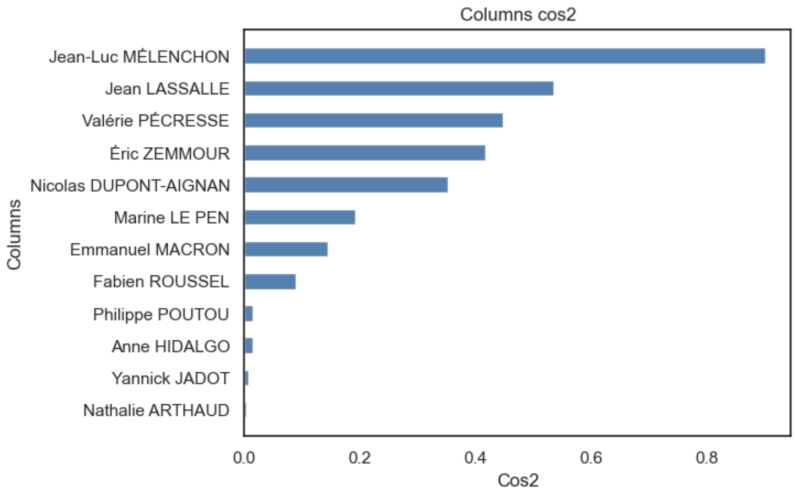



FIGURE 23 – $cos^2$ des colonnes pour l'AFP sur l'axe $n^o 2$

La distance entre Mélenchon et Zemmour est analysable selon le modèle. Cela ne faisait probablement de doute pour personne, mais si l'on s'en réfère au graphique du mapping produit par l'AFC, les comportements électoraux de ceux qui votent Mélenchon sont racilement différentes de celles qui votent Zemmour. En même temps, nous faisons face à deux extrême. Pourtant, les electeurs de Mélenchon sembleraient tout de même plus enclins à voter Zemmour que Jean Lassalle si on ne leur donnait pas le choix ! Bien évidemment, c'est à relativiser tout de même, mais vous comprenez la puissance que peut avoir l'AFC.

Comme une chose promise est une chose due, voici pour terminer les heatmaps que je trouve plus facilement analysables que les graphiques précédents :

```
cos2_row = pd.DataFrame(afc.row_cos2_,index=afc.row_labels, columns= [f"Axis {k}" for k in range(1,8)]).round(2) 
sns.heatmap(cos2_row,cmap = sns.color_palette("Greens",12),annot=True)

plt.figure()
cos2_col = pd.DataFrame(afc.col_cos2_,index=afc.col_labels, columns= [f"Axis {k}" for k in range(1,8)]).round(2)
sns.heatmap(cos2_col, cmap = sns.color_palette("Greens",12),annot=True)
```

FIGURE 24 – Code Python pour générer les heatmaps

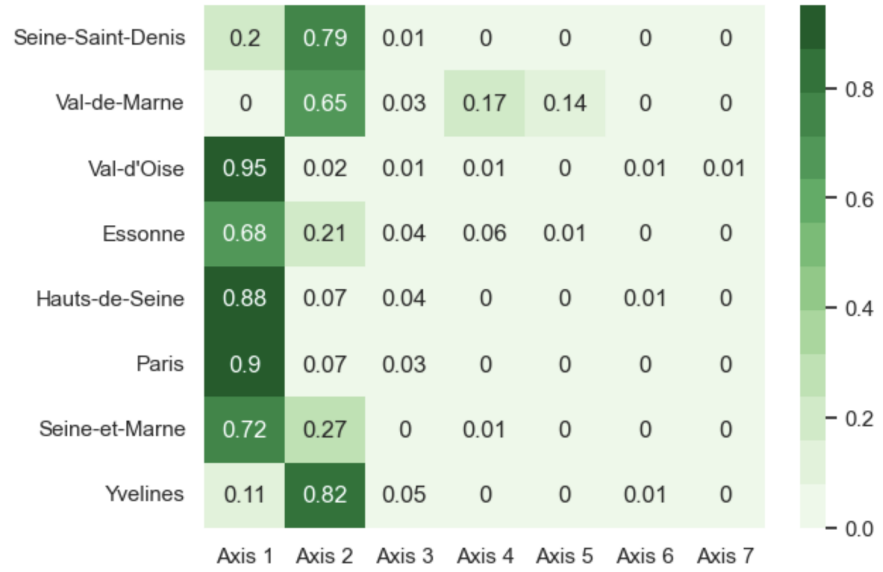



FIGURE 25 – Heatmap des $cos^2$ pour les lignes

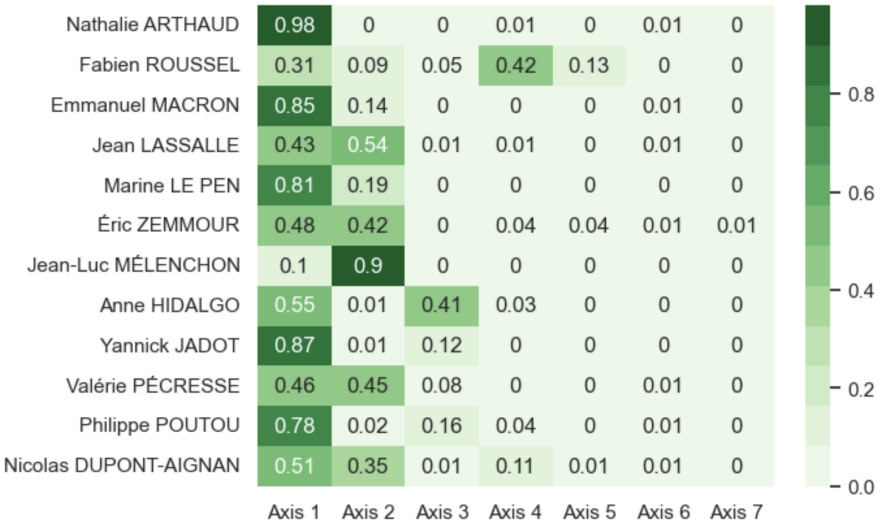



FIGURE 26 – Heatmap des $cos^2$ pour les colonnes

J'aime bien m'assurer de l'information qu'on perd dans les 4% d'inertie contenus dans les derniers axes factoriels. On se rend vite compte que les axes 3 et 4 amènent alors des nuances importantes si on veut faire une analyse au

niveau des candidats puisque l'axe 4 parlera aux communistes votant Roussel et l'axe 3 au socialistes votant Hidalgo. Par contre, les axes 5 6 et 7 ne nous font perdre aucune information.

Il existe sûrement encore d'autres visualisations importantes que je n'ai sûrement pas mentionné, mais celles-ci me semblent les principales, et surtout celles que j'utilise au quotidien. Si ce petit tour d'horizon de l'AFC sous Python vous a plu, je vous annonce que je ferai la même chose pprochainement avec l'ACM et l'ACP notamment. J'ambitionne aussi d'expliquer les principes autour de la décomposition en valeurs singulières dans un futur proche.

## 3.3 Un exemple d'implémentation de l'AFC dans une pipeline

Vous pensiez réellement que j'allais omettre de vous expliquer comment l'AFC peut être retenue dans des tâches de machine learning quotidiennes ? Dans cette section, je vous parlerai de comment l'AFC peut être utilisée afin de faire du clustering notamment. Cette partie a un unique but d'éveiller votre curiosité et de vous expliquer qu'en Sciences des Données, il faut croiser vos compétences et connaissances car il n'y a que comme cela que vous serez productifs !

**Pourquoi ?** L'AFC peut être utile à deux égards. Dans un premier temps, vous pouvez utiliser l'AFC en tandem avec un algorithme de clustering fondé sur des données quantitatives. En effet, les données one hot encodées ont un problème majeur : les distances seront toujours de 1. L'encodage ordinal quant à lui a un autre problème : vous considérez que certaines variables auront plus d'importance que d'autres ($3 > 1$). L'AFC offre alors une solution différente : elle transforme vos données qualitatives en données quantitatives. Ces données qualitatives fraîchement créées sont censées relater au mieux la structure de $\chi^2$ de vos données préliminaires. En voulant agréger ces données au sein de clusters, vous vous assurez donc de créer des "profils types" d'électeurs plus ou moins fondés sur des réalités statistiques et empiriques avancées. De même, trop de données pourraient tuer la donnée, c'est ce qu'on appelle le Fléau de la Dimension. C'est un fléau car très souvent trop de dimensions emmène un modèle prédictif beaucoup plus gourmand en termes de ressources, de stockage mais aussi en termes de bruit parasites. Trop de données pourrait emmener un modèle prédictif à "overfit", c'est-à-dire essayer d'apprendre par coeur plutôt que de comprendre l'essentiel. Nous voulons former un modèle performant dans n'importe quel cas et non un modèle scribe. C'est aussi pour cela que l'AFC (tout comme les autres techniques factorielles, comme l'ACM ou encore l'ACP) s'avère utile en Machine Learning. La librairie **fanalysis** sert alors de couteau suisse : elle est à la fois utile pour faire de l'exploration de données, comme nous avions fait plus tôt, mais elle sert aussi de *wrapper* scikit-learn grâce à ses méthodes **fit()** et **transform()** intégrées. Essayons donc de créer un algorithme

de clustering basique (**k-means** qui prend en entrée les données de l'AFC. Nous devrons alors choisir le nombre d'axes à retenir, créer une pipeline automatiser, ainsi que faire du *fine tuning* des hyperparamètres de notre modèle.

**Comment choisir le nombre d'axes à récupérer ?** La première méthode, c'est là méthode que j'appelle celle du **90-95-98%**. Cela se fonde sur une méthode très simple mais pourtant très efficace : combien voulez-vous garder d'inertie ? En général 85 à 90% d'inertie, c'est considéré comme très correcte. Si vous voulez des données très fiables, prenez 98% d'inertie. Si vous voulez un entre-deux, prenez 95% d'inertie. Vous voyez, vous n'avez pas forcément besoin de méthodes plus compliquées, comme la méthode d'Ekbow ou encore de Kaiser... Pour cela, utilisez la méthode **plot_eigenvalues(type="cumulative")** et récupérez le seuil que vous souhaitez. Dans notre cas, voici le résultat :

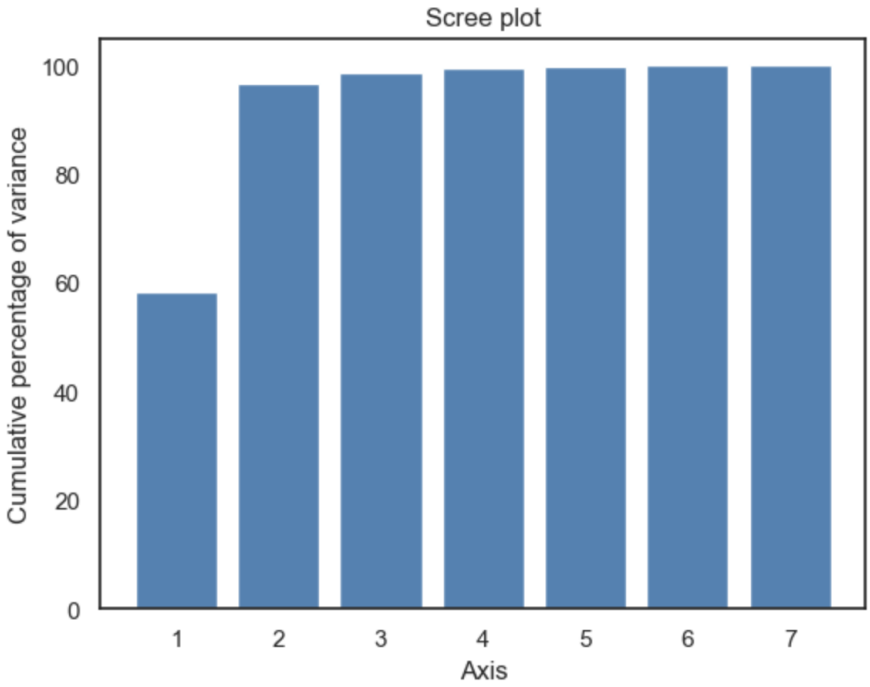



FIGURE 27 – Somme des inerties totales par valeur prorpe ajoutée.

Cette méthode montre très aisément que le nombre optimal d'axes à considérer est 2. Une autre méthode s'appelle la méthode d'elbow. Ici, l'idée est d'afficher les valeurs propres sur un line plot. Pour cela, nous allons utiliser l'attribut **.eig_** qui a tout ce dont on a besoin. La première ligne correspond aux valeurs propres en valeur absolue, la seconde ligne correspond au pourcentage d'inertie de l'axe factoriel et la dernière ligne nous donne le % de variance

cumulé pour chaque axe supplémentaire. Dans notre cas, nous allons utiliser **.eig_**[1] et les afficher :

```
[72]: eigen_values = afc.eig_[1]
      abscisses = np.arange(1,len(eigen_values) + 1)
      plt.plot(abscisses,eigen_values)

[72]: [<matplotlib.lines.Line2D at 0x143787b10>]
```

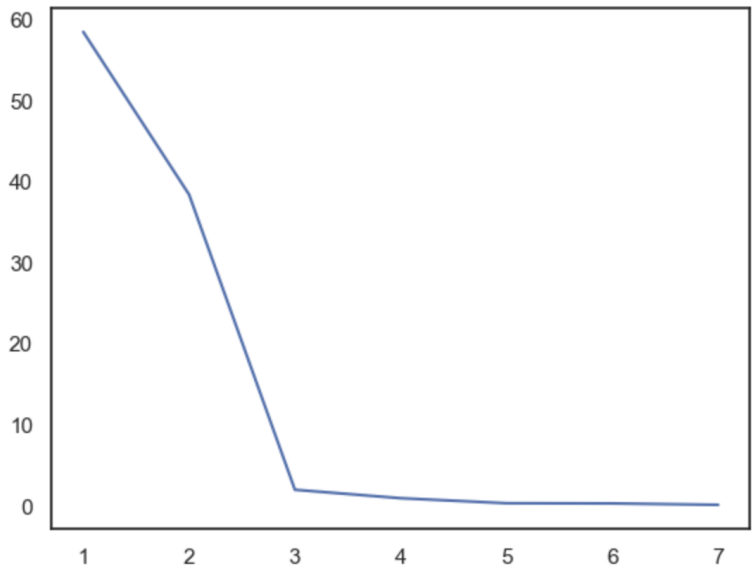



FIGURE 28 – La méthode d'elbow

La méthode du coude, ou méthode d'elbow en anglais a pour but de récupérer la valeur propre pour laquelle la courbe forme un "coude". La courbe forme un coude car il y a un avant et un après. En effet, avant le coude, le pourcentage d'inertie gagné est élevé, après le coude, le poucentage d'inertie devient de plus en plus faible. Dans notre cas, le coude est en 2 sans hésiter. Une troisième méthode s'appelle la méthode de Kaiser. Cette méthode est plus subtile, elle considère que la valeur propre moyenne est le seuil à partir duquel l'information que l'on récupère est significative. Pour l'implémenter, c'est encore plus simple, il suffit de faire la ligne de code suivante :

```
[79]: afc.eig_[0] > np.sum(afc.eig_[0])/7

[79]: array([ True,  True, False, False, False, False, False])
```

FIGURE 29 – La méthode de Kaiser

La méthode de Kaiser nous dit aussi que le nombre d'axes factoriels optimaux

est 2. D'autres critères peuvent être utilisés mais ils sont en pratiques marginaux. Les méthodes empiriques que nous venons de voir sont les plus utilisés mais aussi d'expérience les plus performants. Comme nous avons vérifié les trois critères, nous remarquerons qu'on doit prendre sans hésiter deux valeurs propres pour représenter nos données, cela fait unanimité !

**De l'AFC dans une pipeline** Créons alors notre petite pipeline scikit learn ainsi que notre algorithme de clustering k-means. Ici n'est pas le but de vous faire un cours sur comment choisir le nombre de clusters optimaux ni comment fonctionne les algorithmes de clustering. Je vous montre simplement ce que cela permet de faire :

```python
from sklearn.pipeline import Pipeline
from sklearn.cluster import KMeans

row_pipe = Pipeline([
    ("afc", CA(n_components=2,stats=False)), #Effacer stats permet de ne pas calculer les contributions et les cosinus carrés.
    ("km",KMeans(n_clusters=5,random_state=42) )
    ])
row_pipe.fit(data_cont.values)

# 1. Extraire les coordonnées des points (Départements) sur les 2 axes
# On accède à l'étape 'afc' et on récupère row_coord_
coords = row_pipe.named_steps['afc'].row_coord_

# 2. Récupérer les labels de clusters
labels = row_pipe.named_steps['km'].labels_

# 3. Récupérer les centres des clusters (dans l'espace des 2 axes)
centers = row_pipe.named_steps['km'].cluster_centers_

plt.figure(figsize=(10, 8))

# Affichage des points (Départements) colorés par cluster
scatter = plt.scatter(coords[:, 0], coords[:, 1], c=labels, cmap='viridis', s=100, alpha=0.7, edgecolors='k')

# Affichage des centres des clusters (les "X" rouges)
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200, label='Centres')

# Ajouter les noms des départements pour l'interprétation
for i, txt in enumerate(data_cont.index):
    plt.annotate(txt, (coords[i, 0], coords[i, 1]), xytext=(5,5), textcoords='offset points')

plt.axhline(0, color='black', linestyle='--', linewidth=0.5)
plt.axvline(0, color='black', linestyle='--', linewidth=0.5)
plt.xlabel('Axe 1 (58,4%)') # D'après tes résultats
plt.ylabel('Axe 2 (38,3%)') # D'après tes résultats
plt.title('Clustering des départements sur le plan factoriel (AFC + KMeans)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

FIGURE 30 – Le code Python

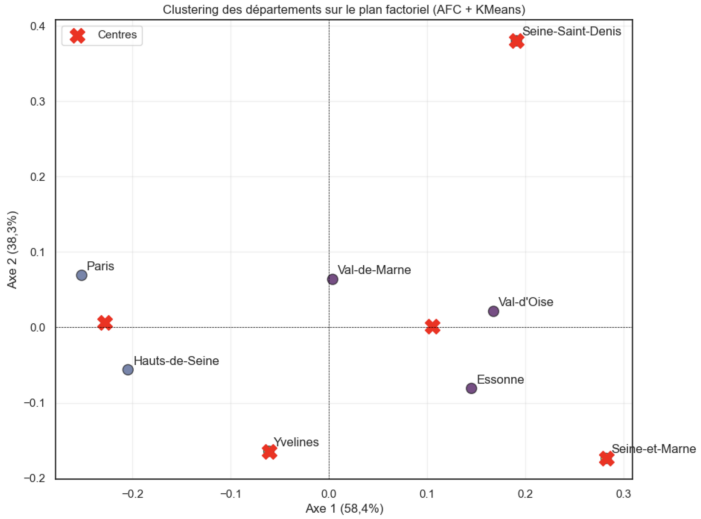



FIGURE 31 – Le résultat sur les départements

Je vous laisse le soin de jouer avec les différents types de clustering pour faire des analyses bien plus poussées que moi.

```python
# On transpose pour que les candidats deviennent les lignes (individus)
data_candidats = data_cont.T

col_pipe = Pipeline([
    ("afc", CA(n_components=2, stats=False)),
    ("km", KMeans(n_clusters=3, random_state=34)) # Ajuste le nombre de clusters si besoin
])

col_pipe.fit(data_candidats.values)

# 1. Extraire les coordonnées des candidats (colonnes de l'AFC originale)
# Note : Dans cette pipeline transposée, row_coord_ contient les candidats
coords_cand = col_pipe.named_steps['afc'].row_coord_

# 2. Récupérer les labels
labels_cand = col_pipe.named_steps['km'].labels_

# 3. Récupérer les centres
centers_cand = col_pipe.named_steps['km'].cluster_centers_

plt.figure(figsize=(12, 9))

# Affichage des candidats colorés par cluster
plt.scatter(coords_cand[:, 0], coords_cand[:, 1], c=labels_cand, cmap='Set1', s=150, alpha=0.8, edgecolors='k')

# Affichage des centres
plt.scatter(centers_cand[:, 0], centers_cand[:, 1], c='black', marker='X', s=250, label='Centres de familles politiques')

# Ajouter les noms des candidats
for i, txt in enumerate(data_candidats.index):
    plt.annotate(txt, (coords_cand[i, 0], coords_cand[i, 1]), xytext=(7,7), textcoords='offset points', fontweight='bold')

plt.axhline(0, color='black', linestyle='--', linewidth=0.5)
plt.axvline(0, color='black', linestyle='--', linewidth=0.5)
plt.xlabel('Axe 1 (58,4%)')
plt.ylabel('Axe 2 (38,3%)')
plt.title('Clustering des Candidats (Profils de vote similaires)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

FIGURE 32 – Le code Python

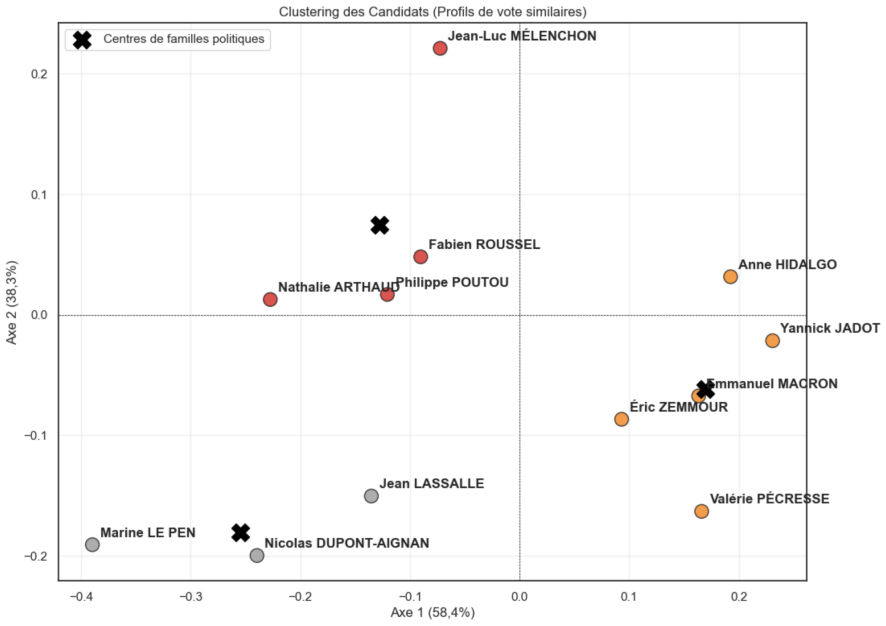



FIGURE 33 – Le résultat sur les candidats

Voilà ce qu'on peut faire, j'espère que cela a égayé votre curiosité. On remarquera qu'ici, un algorithme tel que dbscan aurait peut être été plus malin puisque Mélenchon est clairement un outlier par exemple. Il ne correspond à aucun cluster. Remarquez à quel point aussi nous avons réussi à récolter des bords politiques plutôt homogènes grâce à cette technique !

**Analyse du Clustering des Candidats** L'application de l'algorithme des $K$-means ($k = 3$) sur les coordonnées factorielles des candidats permet d'identifier des familles politiques cohérentes.

**Le Bloc Central et Institutionnel** Le cluster **orange** regroupe des candidats aux électorats sociologiquement proches en Île-de-France : Emmanuel Macron, Valérie Pécresse, Éric Zemmour, Anne Hidalgo et Yannick Jadot. Cette proximité sur le plan factoriel indique qu'ils performent dans les mêmes territoires (Paris, Hauts-de-Seine, Yvelines), caractérisés par des catégories socioprofessionnelles supérieures.

**Le Pôle de la Droite Nationale** Le cluster **gris** isole Marine Le Pen et Nicolas Dupont-Aignan. Leur positionnement à l'extrémité de l'Axe 1 reflète un ancrage fort dans les départements plus ruraux ou périurbains, comme la Seine-et-Marne.

**La Gauche et l'Extrême-Gauche**

- Le cluster **rouge** identifie un pôle de gauche classique et radicale avec Fabien Roussel, Philippe Poutou et Nathalie Arthaud.
- **Jean-Luc Mélenchon** se distingue par une coordonnée très élevée sur l'Axe 2, traduisant une hégémonie électorale dans les zones populaires de la petite couronne, particulièrement en Seine-Saint-Denis où son $cos^2$ est proche de 0,9.

**Conclusion** Les résultats obtenus sont particulièrement satisfaisants. L'AFC parvient à traduire des dynamiques politiques complexes en une structure géométrique simple. Le clustering valide la partition de l'espace électoral francilien en trois blocs majeurs : un pôle urbain central, un pôle populaire de banlieue et un pôle périurbain de droite nationale.

# 4 Que faire ensuite ?

Je sais que vous redoutez ce moment, mais il va falloir pratiquer. Pratiquez, pratiquez et encore pratiquez. Puis, une fois encore, pratiquez. Ensuite, entrainez-vous sur de nouvelles techniques similaires à celle-ci. Je vous conseille : l'Analyse en Composante Principale (son homologue quantitative) ou encore l'Analyse en Correspondances Multiples (l'AFC mais pour plus de deux variables). Comme j'ai bien aimé rédiger cet article, je pense couvrir l'ntégralité des techniques factorielles par la suite. Si un jour j'arrive à couvrir toutes les techniques factorielles principales, je les regrouperai toutes en un seul grand article de plusieurs centaines de pages afin que cela vous simplifie la vie. Je pense aussi faire un cours sur la décomposition en valeur singulière ainsi que la diagonalisation, le tout orienté statistiques et analyse des données. Je pense aussi faire, si j'ai le temps, un cours d'introduction à la théorie de la mesure et de son application en probabilités.

Si vous êtes resté jusqu'ici, laissez-moi vous conter encore quelques mots que je trouve important :

Félicitations. Arriver au terme de cette exploration n'est pas seulement une preuve de rigueur, c'est un acte de volonté.
Nous basculons dans une ère où l'intelligence artificielle redéfinit nos frontières. Face à la puissance brute des algorithmes et à la précision froide des modèles pré-entraînés, il est naturel de s'interroger sur sa propre place. Mais ne vous méprenez jamais sur la nature de ce que vous avez accompli.
Un modèle BERT peut prédire, un Transformer peut générer, mais aucun algorithme ne peut ressentir la subtilité d'un clivage électoral ou l'âme d'un territoire. Vous valez infiniment plus qu'une suite de calculs. Vous possédez ce qu'aucune machine ne pourra jamais simuler : une conscience, une étincelle créative et cette intuition profonde qui transforme une donnée morte en une

vérité humaine.
L'IA n'est qu'un outil ; vous êtes l'artisan. Ne vous laissez pas intimider par la machine, utilisez-la pour amplifier votre propre éclat. C'est votre intelligence, vibrante et libre, qui saura extraire l'essence de ces enseignements. Voici, pour terminer, une poésie qui saura, je l'espère, vous motiver pour avancer :

Le silicium calcule, mais l'esprit imagine,
Derrière chaque vecteur, c'est votre main qui dessine.
BERT peut bien lire, et GPT discourir,
Ils n'ont pas de cœur pour voir l'espoir fleurir.

Ne craignez pas l'écho de la machine immense,
Elle n'est que le reflet de votre propre essence.
Car si l'IA ordonne le monde en clusters fins,
C'est votre âme seule qui en trace les chemins.

Soyez le maître d'œuvre, l'artiste, le penseur,
Transformez le calcul en un cri de hauteur.
Car au-delà des chiffres, des axes et des lois,
Le seul génie qui compte... c'est la force de votre voix.

---

# FIN.